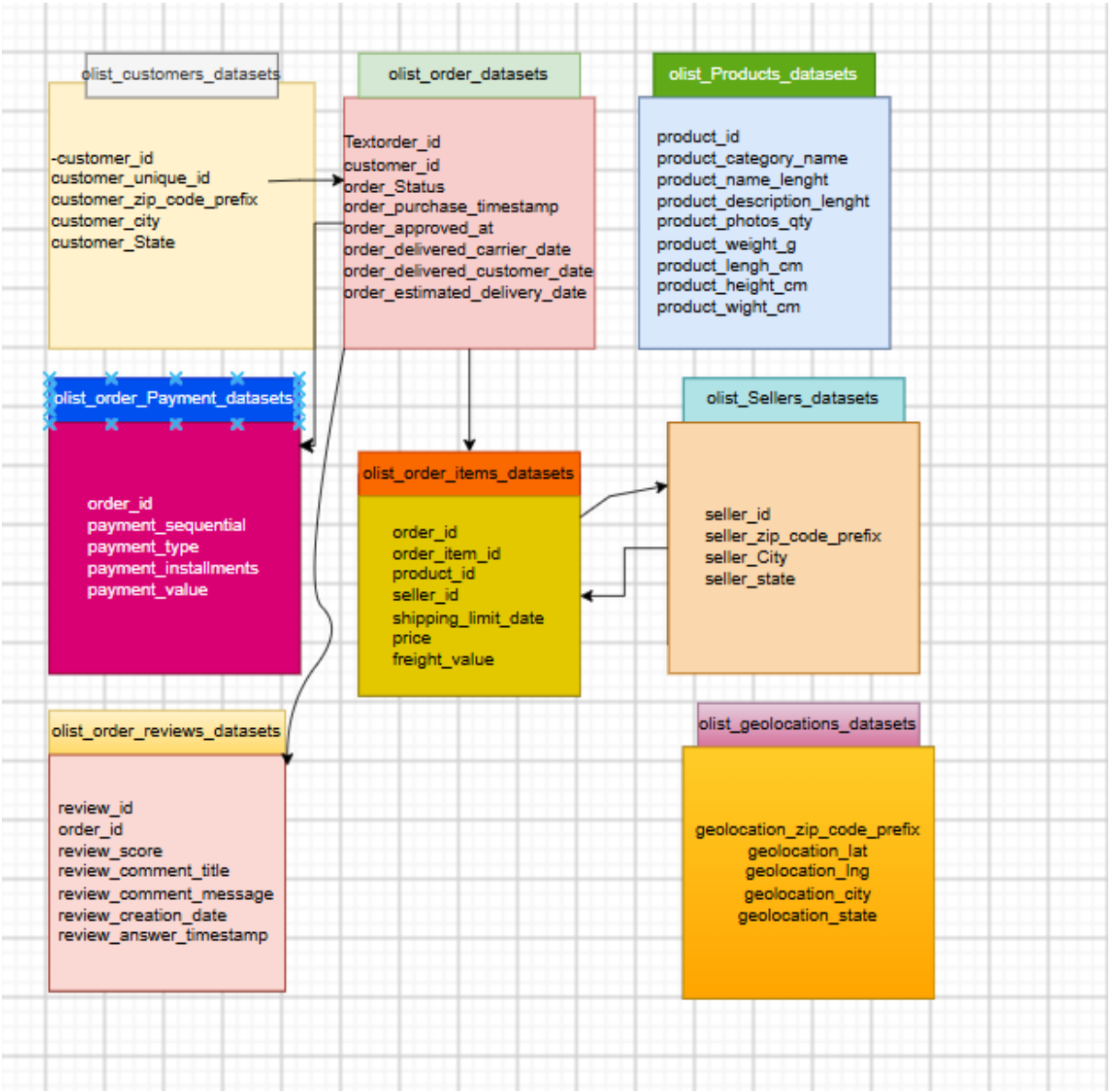
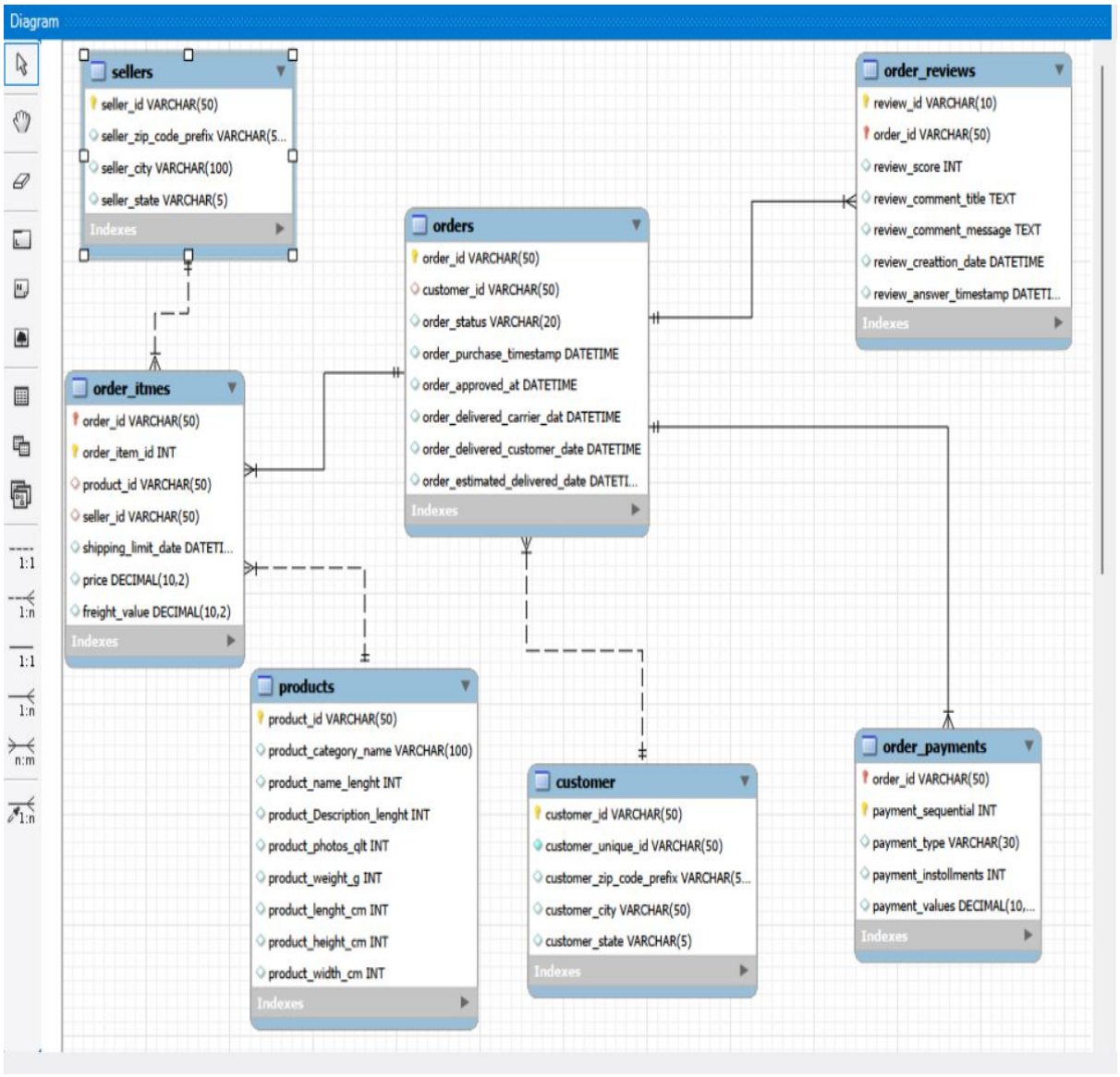
ASSIGNMENT 1: SQL MASTERY - The E-Commerce Analytics Challenge

Part A :-Database Design & Data Quality

1. **Design a normalized database schema (3NF) with ERD showing all relationships between 8 tables.**




Diagram
sellers
seller_id VARCHAR(50)
seller_zip_code_prefix VARCHAR(5...
seller_city VARCHAR(100)
seller_state VARCHAR(5)
Indexes
order_reviews
review_id VARCHAR(10)
order_id VARCHAR(50)
review_score INT
review_comment_title TEXT
review_comment_message TEXT
review_creattion_date DATETIME
review_answer_timestamp DATETI...
Indexes
orders
order_id VARCHAR(50)
customer_id VARCHAR(50)
order_status VARCHAR(20)
order_purchase_timestamp DATETIME
order_approved_at DATETIME
order_delivered_carrier_dat DATETIME
order_delivered_customer_date DATETIME
order_estimated_delivered_date DATETI...
Indexes
order_itmes
order_id VARCHAR(50)
order_item_id INT
product_id VARCHAR(50)
seller_id VARCHAR(50)
shipping_limit_date DATETI...
price DECIMAL(10,2)
freight_value DECIMAL(10,2)
Indexes
products
product_id VARCHAR(50)
product_category_name VARCHAR(100)
product_name_lenght INT
product_Description_lenght INT
product_photos_qlt INT
product_weight_g INT
product_lenght_cm INT
product_height_cm INT
product_width_cm INT
Indexes
customer
customer_id VARCHAR(50)
customer_unique_id VARCHAR(50)
customer_zip_code_prefix VARCHAR(5...
customer_city VARCHAR(50)
customer_state VARCHAR(5)
Indexes
order_payments
order_id VARCHAR(50)
payment_sequential INT
payment_type VARCHAR(30)
payment_instollments INT
payment_values DECIMAL(10,...
Indexes
1:1
1:n
1:1
1:n
n:m
1:n

Question :2.Identify and document 10+ data quality issues in the raw CSV files.

1. Find missing customer_id

```
8   -- 1.Find missing customer_id
9   select count(*) as null_customer_id
10  from customers
11  where customer_id is null
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

null_customer_id
0

2 Find duplicate customer_unique_id

```
14  -- 2 find duplicate customer_unique_id
15
16  select customer_unique_id , count(*) as duplicate_value
17  from customers
18  group by customer_unique_id
19  having count(*) >1
20
21
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

customer_unique_id	duplicate_value
4c93744516667ad3b8f1fb645a3116a4	2
1175e95fb47ddff9de6b2b06188f7e0d	2
295c05e81917928d76245e842748184d	2
e9dd12dca17352644a959d9dea133935	2
e079b18794454de9d2be5c12b4392294	2
c9158d089637ab443c78984d20da7fc0	2
7a380cb5434e6b6b5b37d45bb99dbe8a	2
13ac6da47fef1b17b16f511e03b57918	2

Result 6

3 Issue

```
22  -- find orphane_order
23
24  select count(*) as orphane_order
25  from orders o
26  left join customers c
27  on c.customer_id  = o.customer_id
28  where c.customer_id is null
29
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

orphane_order
1182

4 Issue Find the negative price

```
30      -- find negative price
31      select * from order_items;
32
33      select count(*) as negative_price
34      from order_items
35      where price < 0
36
37
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

negative_price
0

5 Issue

```
33      -- Reviews is null reviews
34      select * from order_reviews
35
36      select count(*) as null_reviews
37      from order_reviews;
38      where  review_score is null
39
40
```

esult Grid | Filter Rows: | Export: | Wrap Cell Content:

null_reviews
298

Result Grid | Form Editor | Read Only

esult 32 ×

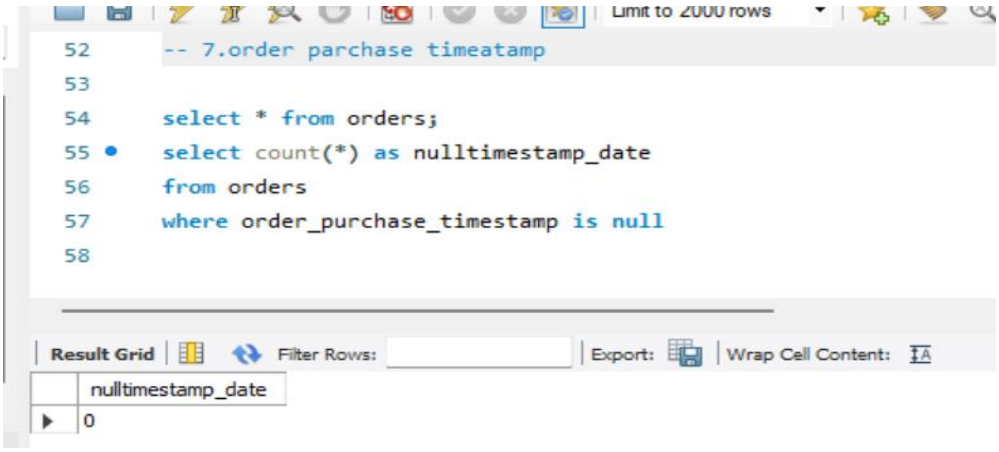
6 Issue Missing Category name

```
44      select * from products
45      -- missing category name
46
47      select count(*) as missing_category_name
48      from products
49      where product_category_name is null
50
```

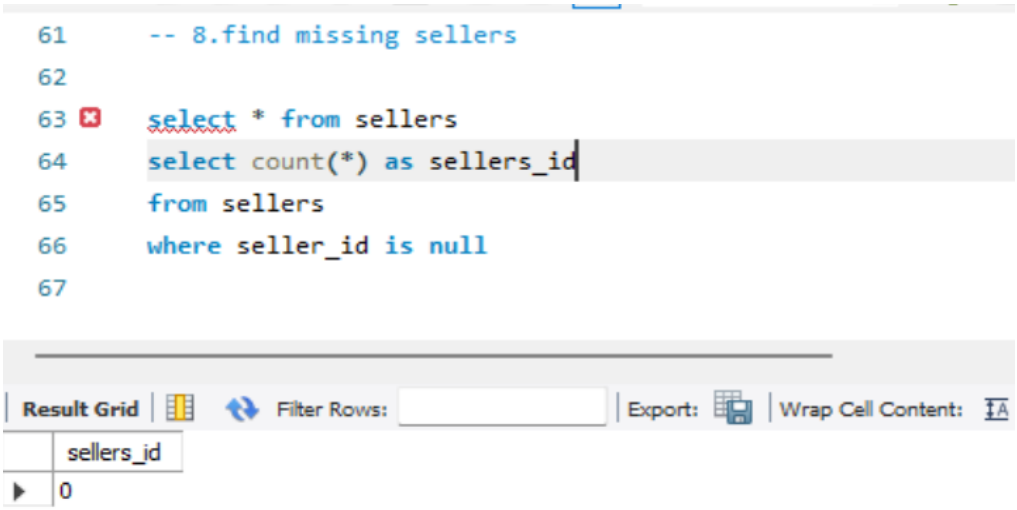
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

missing_category_name
0

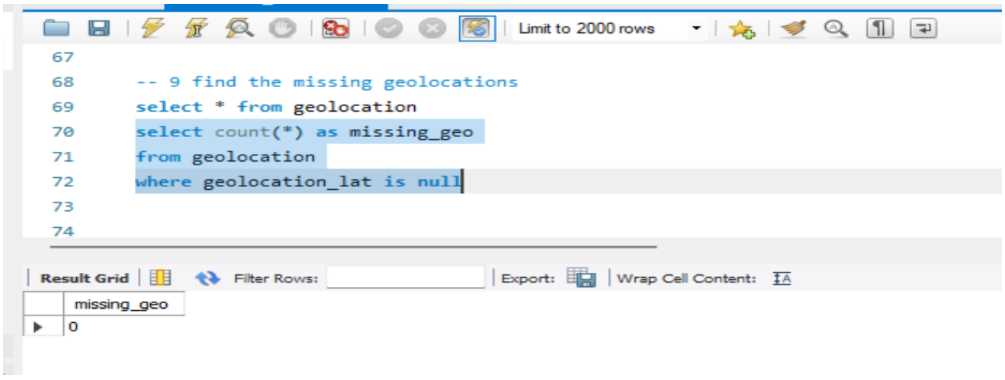
7 Issue find order Purchase timestamp is null or not



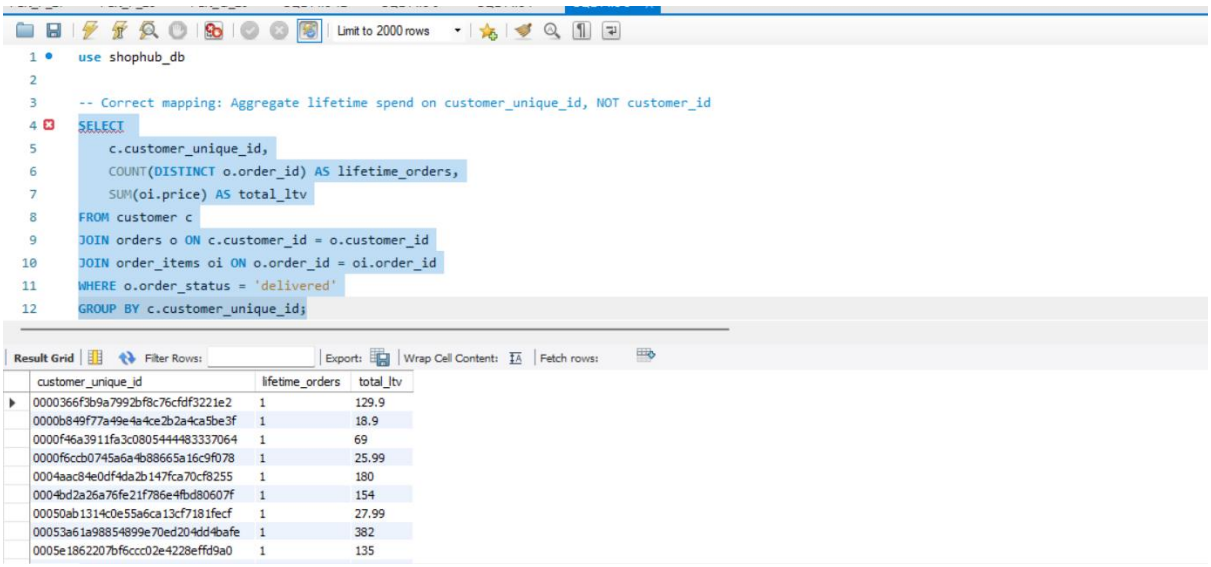
8 Issue Find missing sellers



9 Find missing geolocations



10 Correct mapping: Aggregate lifetime spend on customer_unique_id, NOT customer_id



```
use shophub_db

-- Correct mapping: Aggregate lifetime spend on customer_unique_id, NOT customer_id
SELECT
    c.customer_unique_id,
    COUNT(DISTINCT o.order_id) AS lifetime_orders,
    SUM(oi.price) AS total_ltv
FROM customer c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.order_status = 'delivered'
GROUP BY c.customer_unique_id;
```

Result Grid

customer_unique_id	lifetime_orders	total_ltv
0000366f3b9a7992bf8c76cfdf3221e2	1	129.9
0000b849f77a49e4a4ce2b2a4ca5be3f	1	18.9
0000f46a3911fa3c0805444483337064	1	69
0000f6ccb0745a6a4b88665a16c9f078	1	25.99
0004aac84e0df4da2b147fca70cf8255	1	180
0004bd2a26a76fe21f786e4fbd80607f	1	154
00050ab1314c0e55a6ca13cf7181fecf	1	27.99
00053a61a98854899e70ed204dd4bafe	1	382
0005e1862207bf6ccc02e4228effd9a0	1	135

Question :4- Detection and Remove of Duplicate Customers

1.Objective

To Identify duplicate customer entries based on customer_unique_id and remove older Records while preserving the most recent customer record in the customer table

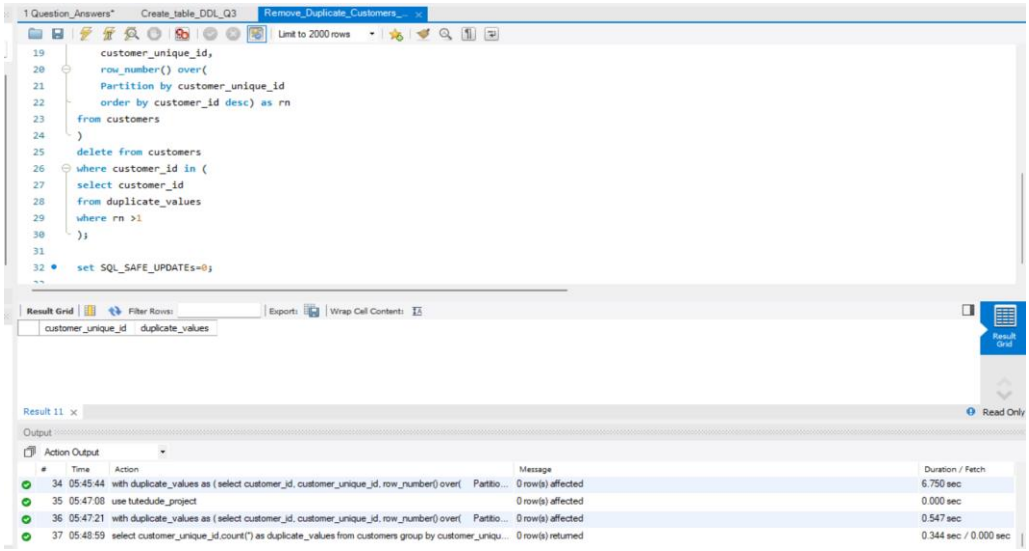
2.SQL Implementation Script

Detect and Remove Duplicate Values

```
use tutedude_project
select * from customers;
select customer_unique_id,count(*) as duplicate_values
from customers
group by customer_unique_id
having count(*) >1;
-- Remove Duplicate
with duplicate_values as
(
```

```
Select customer_id, customer_unique_id,
        row_number() over(
    Partition by customer_unique_id
    order by customer_id desc) as rn
from customers
)
delete from customers
where customer_id in (
select customer_id
from duplicate_values
where rn >1
);
set SQL_SAFE_UPDATEs=0;
```

3 . Execution Screenshot



4 . Technical Explanation & implementation Notes

-Groping Logic: The Query user customer_unique_id to identify duplicate because individual customers can accidentally generate multiple customer_id entries.

-Safe update handling: MySQL Workbench enables safe update mode by default, which block multi-row DELETE queries without explicit primary key conditions in the WHERE clause. Executing SET SQL_SAFE_UPDATES=0; Temporarily by passer Eroor 1175 and allows successful cleanup

Question :5 Data Dictionary Documentation

1.Overview

This data dictionary provide comprehensive documentation for all 8

Normalized table within the Shophub_db database schema. IT outline the

Structural definition, data types, key constraints and business meanings for each

Field to ensure data governance clarity across development teams

2.Tabale specifications

Table 1: Geolocation

- Business Purpose: Stores geographics locations mapping (Zip code prefixes, latitudes, longitude ,city, state) for logistics and address validation .

Column Name	Data Type	Constraints	Primary key/Foreign Key	Business/description
geolocation_zip_code_prefic	varchar(10),	Not Null	-	First 5 digits of the Postal ZIP Code
geolocation_lat	decimal(10,8),	Null	-	Geographic Latitude Coordinate
geolocation_lng	decimal(10,8),	Null	-	Geographic longitude coordinate
geolocation_city	varchar(100),	Not Null	-	City name associated With the Zip code
geolocation_state	varchar(50)	Not Null	-	State abbreviation/Code

Table 2 Customer

Business Purpose :Maintains master demographic detail of registered e-commerce customers .

Column Name	Data Type	Constraints	Primary/Foreign Key	Business Meaning / Description

customer_id	VARCHAR(50)	NOT NULL	PRIMARY KEY	Unique identifier for each order-level customer entity
customer_unique_id	VARCHAR(50)	NOT NULL	-	Permanent unique identifier for the actual individual customer
customer_zip_code_prefix	VARCHAR(50)	NULL	-	Customer location postal code prefix
customer_city	VARCHAR(50)	NOT NULL	-	City where the customer is located
customer_state	VARCHAR(5)	NOT NULL	-	State code where the customer resides

Table 3: Sellers

Business purpose :Stores merchant/seller registration details responsible for fulfilling

Order items

Column Name	Data Type	Constraints	Primary/Foreign Key	Business Meaning / Description
seller_id	VARCHAR(50)	NOT NULL	PRIMARY KEY	Unique identifier for each registered seller
seller_zip_code_prefix	VARCHAR(50)	NULL	-	Seller's primary postal code prefix
seller_city	VARCHAR(100)	NOT NULL	-	City where seller operating hub is located
seller_state	VARCHAR(5)	NOT NULL	-	State code of seller hub

Table 4 : Products

Business Purpose: catalog of items is available for sale along with category specification and physical dimension for shipping calculations

Column Name	Data Type	Constraints	Primary/Foreign Key	Business Meaning / Description
product_id	VARCHAR(50)	NOT NULL	PRIMARY KEY	Unique identifier for each product stock item
product_category_name	VARCHAR(100)	NULL	-	Classification category of the product
product_name_length	INT	CHECK (>= 0)	-	Text character count of product title
product_description_length	INT	CHECK (>= 0)	-	Text character count of product description
product_photos_qlt	INT	CHECK (>= 0)	-	Total count of published product catalog photos
product_weight_g	INT	CHECK (>= 0)	-	Product package weight in grams
product_length_cm	INT	CHECK (>= 0)	-	Package length measurement in centimeters
product_height_cm	INT	CHECK (>= 0)	-	Package height measurement in centimeters
product_width_cm	INT	CHECK (>= 0)	-	Package width measurement in centimeters

Table 5: Orders

Business purpose: central transaction entity recording order status lifecycle timestamps, and customer ownership

Column Name	Data Type	Constraints	Primary/Foreign Key	Business Meaning / Description
order_id	VARCHAR(50)	NOT NULL	PRIMARY KEY	Unique reference code for the order transaction
customer_id	VARCHAR(50)	NOT NULL	FOREIGN KEY	Links order to customer(customer_id)
order_status	VARCHAR(20)	NOT NULL	-	Order lifecycle state (e.g., delivered, shipped, canceled)
order_purchase_timestamp	DATETIME	NOT NULL	-	Exact timestamp when customer placed the order
order_approved_at	DATETIME	NULL	-	Timestamp when payment approval was completed
order_delivered_carrier_date	DATETIME	NULL	-	Timestamp when order was handed to logistics carrier
order_delivered_customer_date	DATETIME	NULL	-	Actual timestamp when package was delivered to customer
order_estimated_delivered_date	DATETIME	NULL	-	Initial estimated delivery deadline provided to customer

Table 6: Order items

Business Purpose : Line- items breakdown linking individual products, sellers, price, and freight values within an order (M:N junction table).

Column Name	Data Type	Constraints	Primary/Foreign Key	Business Meaning / Description
order_id	VARCHAR(50)	NOT NULL	PRIMARY KEY, FK	Links to orders(order_id)
order_item_id	INT	NOT NULL	PRIMARY KEY	Sequential item index number within the order
product_id	VARCHAR(50)	NOT NULL	FOREIGN KEY	Links item to products(product_id)
seller_id	VARCHAR(50)	NOT NULL	FOREIGN KEY	Links item fulfillment responsibility to sellers(seller_id)
shipping_limit_date	DATETIME	NULL	-	Seller shipping deadline date
price	DECIMAL(10,2)	CHECK (>= 0)	-	Unit price of the individual product item
freight_value	DECIMAL(10,2)	CHECK (>= 0)	-	Freight/shipping charge assigned to the item

Table 7: Order Payments:

Business Purpose : Store payment execution method, installments, and monetary amounts for each order

Column Name	Data Type	Constraints	Primary/Foreign Key	Business Meaning / Description

order_id	VARCHAR(50)	NOT NULL	PRIMARY KEY, FK	Links payment record to orders(order_id)
payment_sequential	INT	NOT NULL	PRIMARY KEY	Sequence index for split or multiple payments
payment_type	VARCHAR(30)	NOT NULL	-	Payment mode (e.g., credit_card, boleto, voucher, UPI)
payment_installments	INT	CHECK (>= 0)	-	Number of payment installment cycles selected
payment_value	DECIMAL(10,2)	CHECK (>= 0)	-	Total monetary amount processed in this transaction

Table 8: order reviews

Business Purpose: Capture customer satisfaction ratings, reviews title, comments ,and answer response timestamps

Column Name	Data Type	Constraints	Primary/Foreign Key	Business Meaning / Description
review_id	VARCHAR(100)	NOT NULL	PRIMARY KEY	Unique identifier for customer review submission
order_id	VARCHAR(50)	NOT NULL	PRIMARY KEY, FK	Links review to orders(order_id)
review_score	INT	CHECK (1 TO 5)	-	Satisfaction rating on a 1-to-5 star scale
review_comment_title	TEXT	NULL	-	Headline text title of the customer review
review_comment_message	TEXT	NULL	-	Detailed text feedback message written by customer
review_creation_date	DATETIME	NULL	-	Timestamp when review survey was sent to customer
review_answer_timestamp	DATETIME	NULL	-	Timestamp when customer submitted their review

Part B: Data Manipulation & Retrieval (5 Questions)

6. Insert cleaned data using transactions and demonstrate COMMIT and ROLLBACK ?

1. Objective & Methodology

The goal of this task is to ensure ACID properties (specifically Atomicity and consistency) during data population is shophub_db . Using SQL truncation Control language (TCL) commands (Start transaction , commit , and rollback), Data is validate before making permanent change to the database

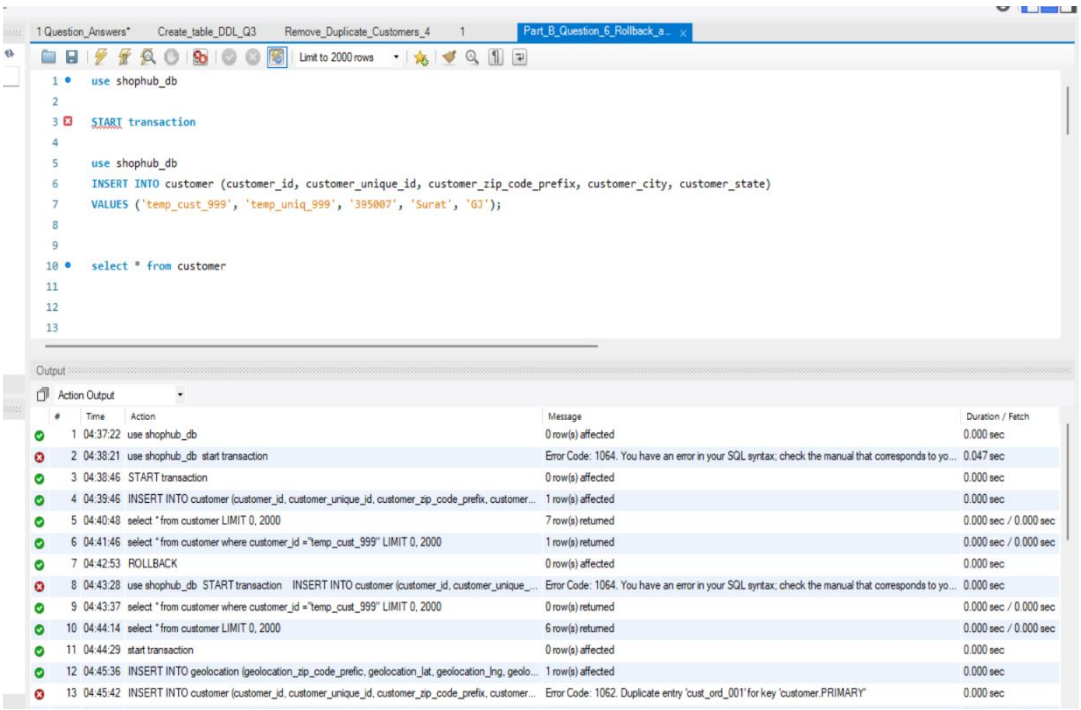
2.Demonstration 1.Testing data integrity via Rollback

. Concept : A transaction is initiated to inset a temporary record (temp_cust_999). A Rollback command is issued to simulate an error handling scenario, reverting uncommitted changes without affecting the underlying database.

• SQL Script Executed:

```
START TRANSACTION;
INSERT INTO customer (customer_id, customer_unique_id,
customer_zip_code_prefix, customer_city, customer_state)
VALUES ('temp_cust_999', 'temp_uniq_999', '395007', 'Surat', 'GJ');
ROLLBACK;
SELECT * FROM customer WHERE customer_id = 'temp_cust_999';
```

. Verification Result: The query return an empty set (0 row), confirming that the uncommitted temporary record was completely reverted by rollback



3. Demonstration 2: Permanent Clean Data Population via COMMIT

• Concept: Cleaned records were inserted into parent tables (geolocation, customer, sellers, products) and child tables (orders, order_itmes, order_payments, order_reviews) in strict foreign key dependency order. A COMMIT command was then executed to permanently save the changes.

• SQL Script Executed:

```
START TRANSACTION;

-- Insert into Parent & Child Tables

INSERT INTO geolocation ... ;

INSERT INTO customer ... ;

INSERT INTO sellers ... ;

INSERT INTO products ... ;

INSERT INTO orders ... ;

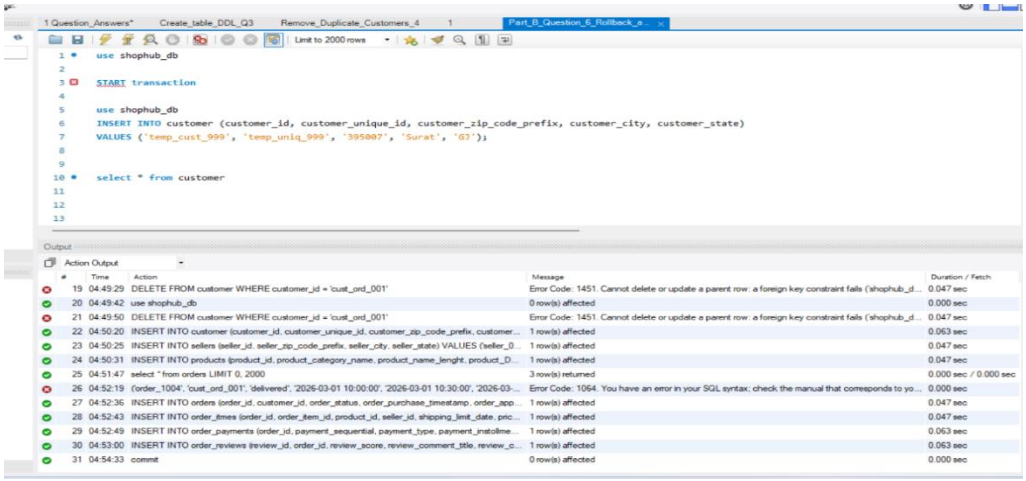
INSERT INTO order_itmes ... ;

INSERT INTO order_payments ... ;

INSERT INTO order_reviews ... ;

COMMIT;
```

• Verification Result: Workbench Action Output panel showed successful execution with green checkmarks for all insertions and the final COMMIT statement.



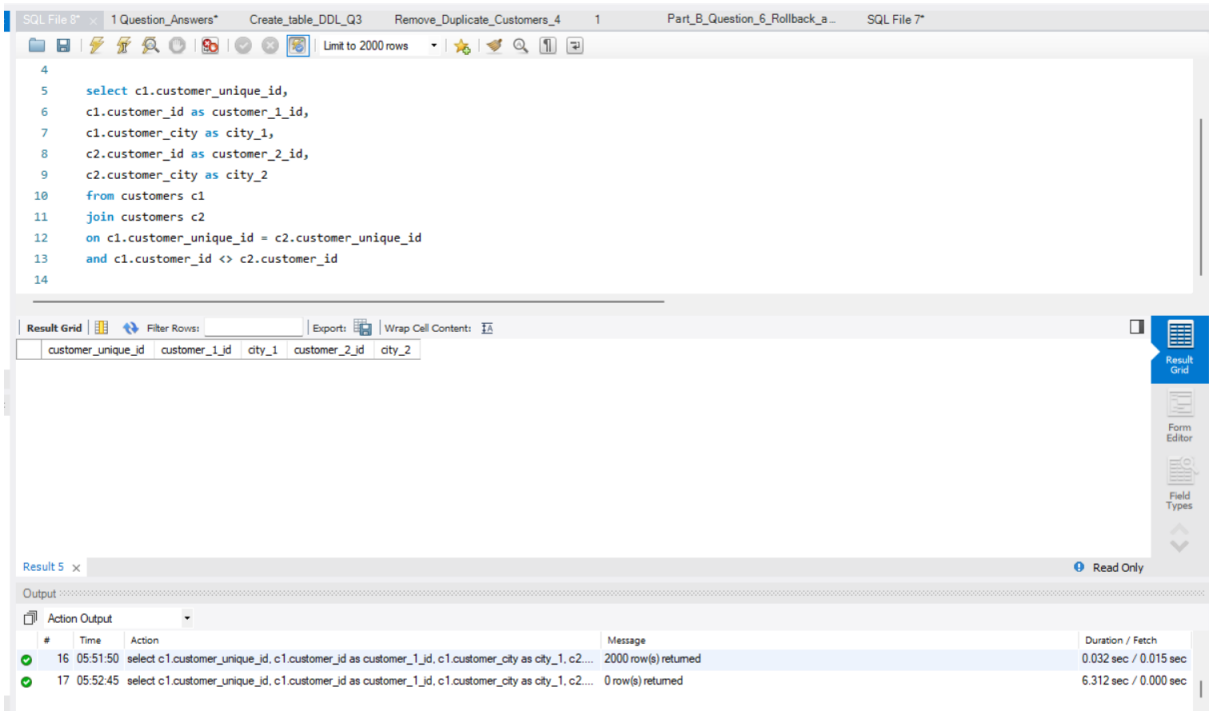
Question 7:Find customers who have the same email but different names/addresses.

1 . Explanation & Approach
To identify potential duplicate accounts or confirmation customer details, a Self join is performed on the "customer" table. The query matches records sharing the same "Customer_unique_id" while having different "customer_id's" or addresses

2 SQL Query executed

```
select c1.customer_unique_id,
c1.customer_id as customer_1_id,
c1.customer_city as city_1,
c2.customer_id as customer_2_id,
c2.customer_city as city_2
from customers c1
join customers c2
on c1.customer_unique_id = c2.customer_unique_id
and c1.customer_id <> c2.customer_id
```

3 Output & conclusion:
The query returned 0 rows (empty set). This confirms high data integrity, indicating duplicate customer accounts with conflicting addresses or multiple distinct customer IDs exist in the system



Question : 8: Identify orphan records referencing non-existent products or orders.

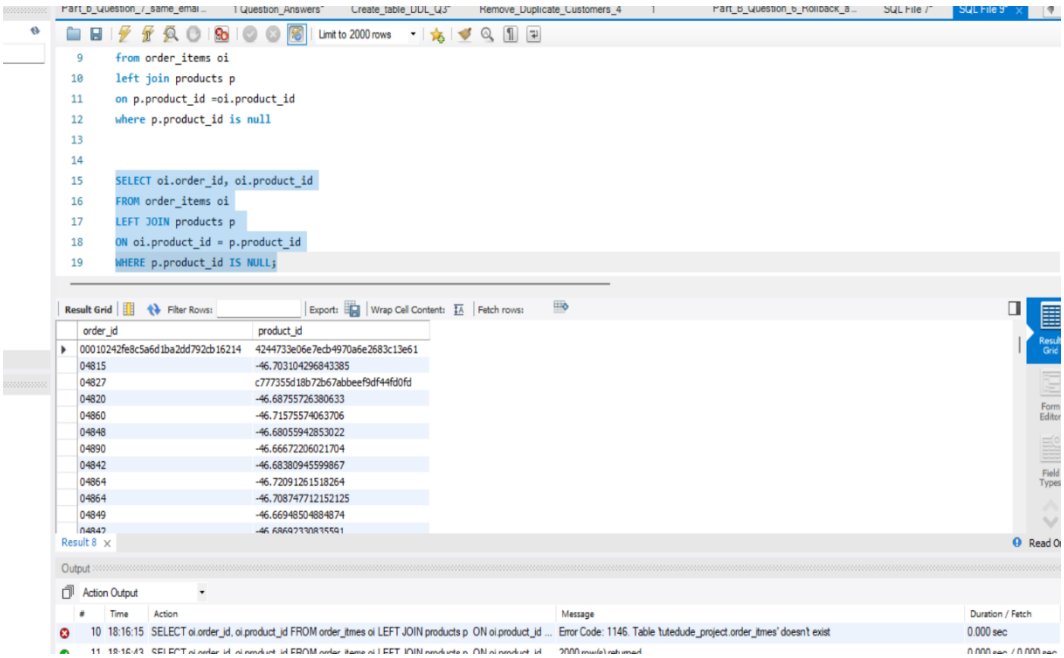
1.Explanation & Approach:

Orphan record occur when child tables contain foreign keys referencing non-existent parent records . A LEFT JOIN is performed between " Order_items " and 'Products', filtered by ' where p.product_id is null', to identify ordered items referencing product IDs missing from the primary catalog.

2.SQL Query Executed:

```
select oi.order_id,oi.product_id
from order_items oi
left join products p
on p.product_id =oi.product_id
where p.product_id is null
```

3. Output



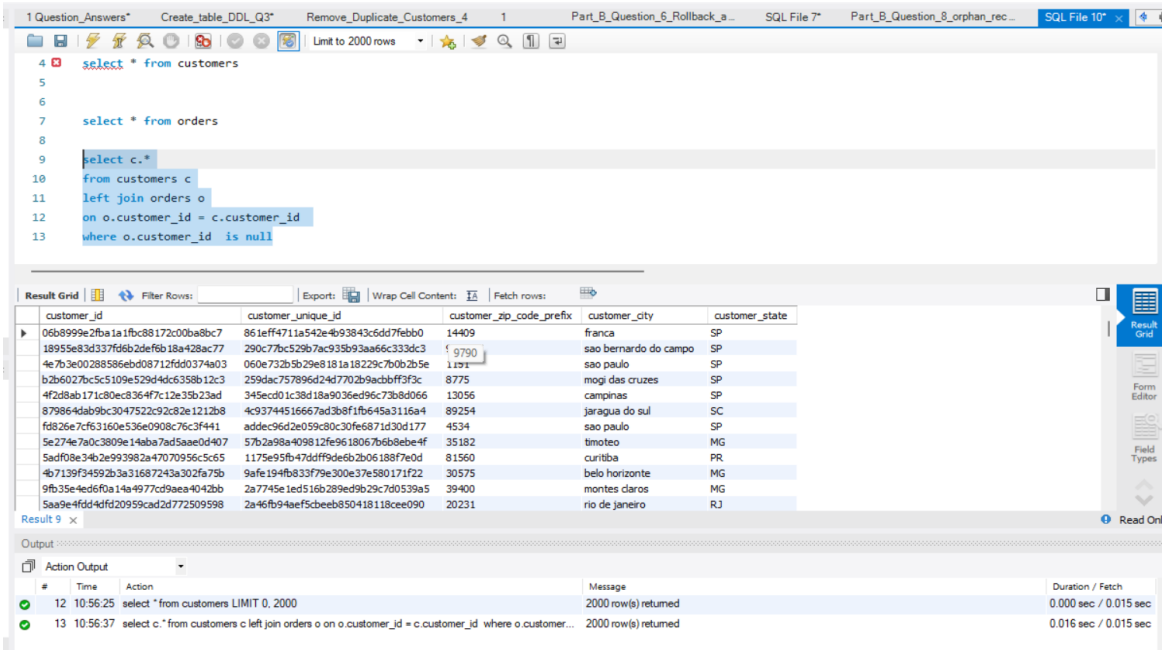
Question 9 :-List customers who registered but never placed an order.

1. Explanation And Approach :
   To identify registered customer who have never place an order,
   A left join is perform between them ‘customer’ table and the ‘orders’ tables on ‘customer_id ’ filtering with ‘WHERE o.customer_id is null’ isolates customer Account that have no corresponding transactions in the orders table.

2. SQL Query Executed:

```
 SELECT  c.*
FROM customer c
 LEFT JOIN orders o
 ON o.customer_id = c.customer_id
WHERE o.customer_id IS NULL;
```

   3 Output & conclusion
   The query executed successfully and returned customer records who have registered accounts but zero associated orders in the system. This identifies inactive user account that can be targeted for re-engagement marketing campaigns



Part B: Question 10 :

Requirement: Detect potential fraud where payment value differs from order total.

1. Explanation & Approach:

To detect potential fraud or financial discrepancies, we compare the total value of items purchased per order against the total payments received for that order:

- Order Total is calculated by aggregating the sum of item prices and shipping costs ('price + freight_value') from the 'order_items' table.

- Payment Total is calculated in a dedicated subquery by summing 'payment_values' from the 'order_payments' table per order.

- Joining the aggregated item totals with the payment subquery prevents Cartesian product errors (multiplied joins) when orders contain multiple items or payment installments.

- The 'HAVING order_total <> p.payment_total' condition filters out matching orders and isolates only mismatched transactions.

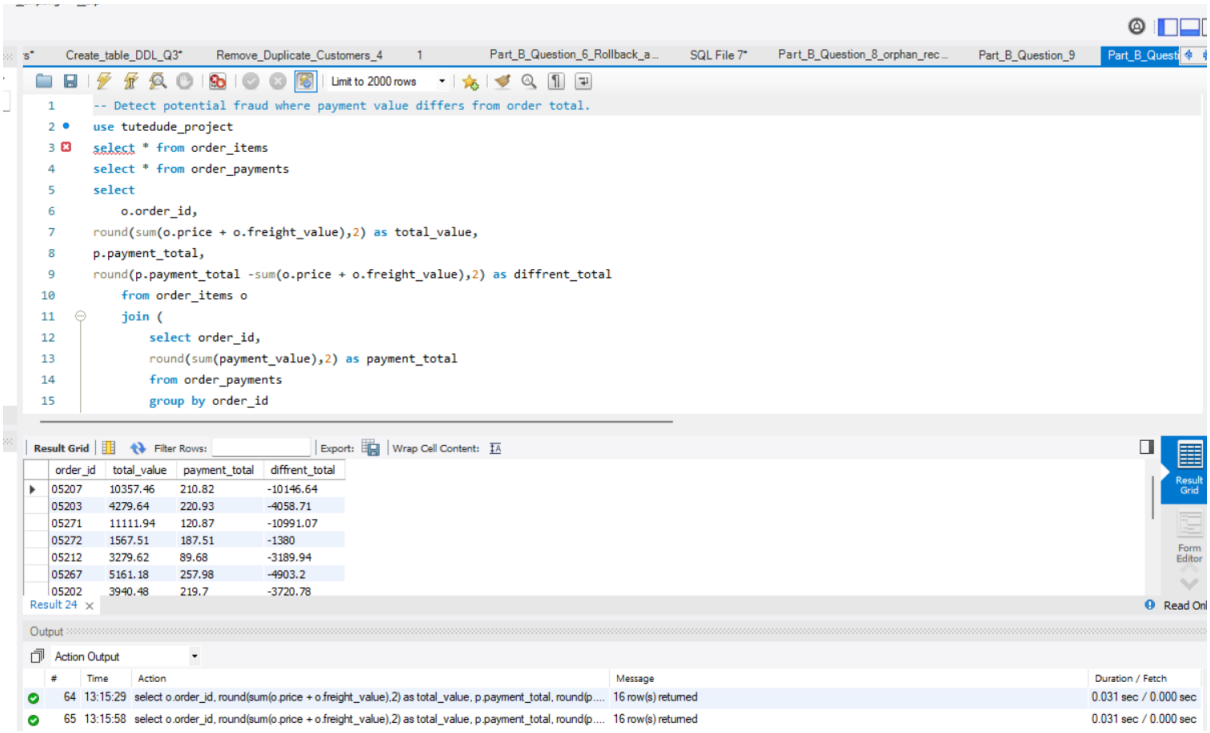
2. SQL Query Executed:

```
SELECT
    o.order_id,
    ROUND(SUM(o.price + o.freight_value), 2) AS order_total,
    p.payment_total,
    ROUND(p.payment_total - SUM(o.price + o.freight_value), 2) AS price_difference
FROM order_items o
JOIN (
    SELECT
        order_id,
        ROUND(SUM(payment_values), 2) AS payment_total
    FROM order_payments
    GROUP BY order_id
) p ON o.order_id = p.order_id
GROUP BY o.order_id, p.payment_total
HAVING order_total <> p.payment_total;
```

3. Output & Conclusion:

[Option A - If rows are returned]:

The query successfully identified mismatched transactions where the processed payment amount does not equal the item pricing total. These records indicate potential transaction fraud, unapplied discounts/vouchers, or technical rounding discrepancies that require financial auditing.



```
-- Detect potential fraud where payment value differs from order total.
use tutedude_project
select * from order_items
select * from order_payments
select
    o.order_id,
round(sum(o.price + o.freight_value),2) as total_value,
p.payment_total,
round(p.payment_total -sum(o.price + o.freight_value),2) as diffrent_total
    from order_items o
    join (
        select order_id,
        round(sum(payment_value),2) as payment_total
        from order_payments
        group by order_id
```

order_id	total_value	payment_total	diffrent_total
05207	10357.46	210.82	-10146.64
05203	4279.64	220.93	-4058.71
05271	11111.94	120.87	-10991.07
05272	1567.51	187.51	-1380
05212	3279.62	89.68	-3189.94
05267	5161.18	257.98	-4903.2
05202	3940.48	219.7	-3720.78

#	Time	Action	Message	Duration / Fetch
64	13:15:29	select o.order_id, round(sum(o.price + o.freight_value),2) as total_value, p.payment_total, round(p....	16 row(s) returned	0.031 sec / 0.000 sec
65	13:15:58	select o.order_id, round(sum(o.price + o.freight_value),2) as total_value, p.payment_total, round(p....	16 row(s) returned	0.031 sec / 0.000 sec

Part C: Complex Aggregations (5 Questions)

11.Calculate monthly revenue with Month-over-Month growth percentage.

1. Explanation & Approach:

To calculate monthly revenue and track financial momentum:

- The 'orders' table is joined with 'order_items' on 'order_id'.

- Due to dataset column-shifting issues in raw order timestamps, valid order completion dates were extracted from 'order_delivered_carrier_date' and aggregated at the year-month level ('%Y-%m').

- Total monthly product revenue is calculated by summing item prices ('SUM(oi.price)').

- A Common Table Expression (CTE) utilizes the 'LAG()' window function to retrieve the prior month's revenue per row.

- Month-over-Month (MoM) Growth Percentage is calculated using:

((Current Month Revenue - Previous Month Revenue) / Previous Month Revenue) * 100.

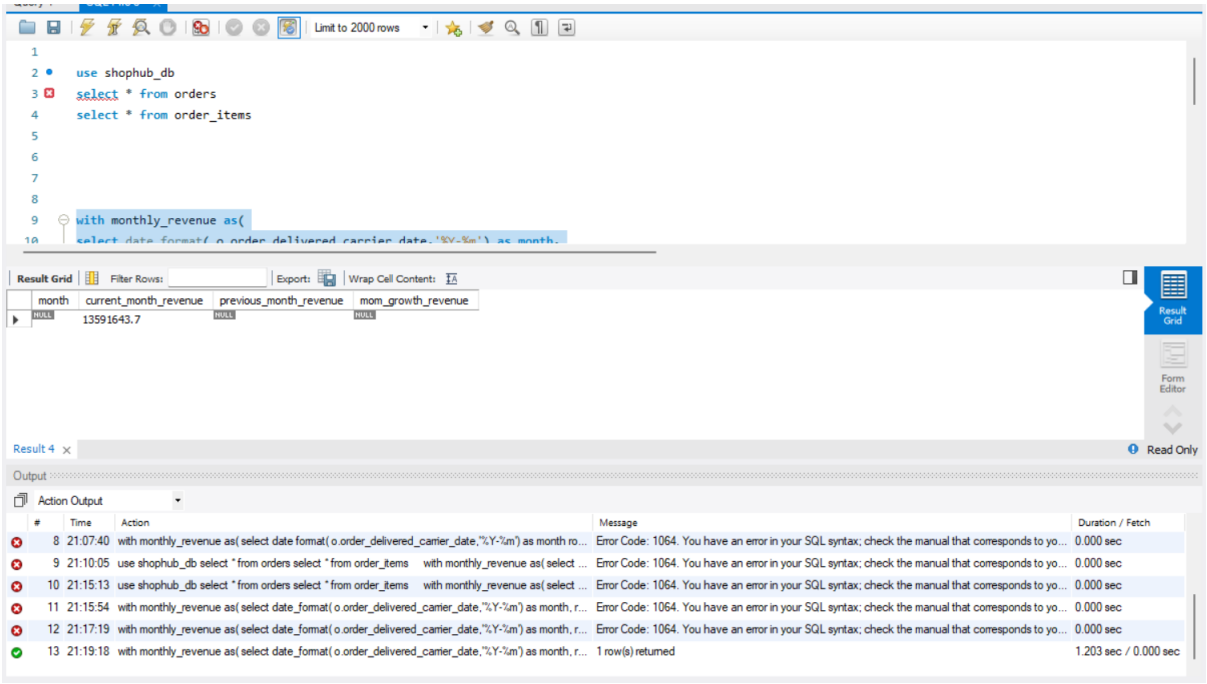
2. SQL Query Executed:

```
WITH monthly_revenue AS (
    SELECT
        DATE_FORMAT(o.order_delivered_carrier_date, '%Y-%m') AS month,
        ROUND(SUM(oi.price), 2) AS current_month_revenue
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    WHERE o.order_delivered_carrier_date IS NOT NULL
    GROUP BY DATE_FORMAT(o.order_delivered_carrier_date, '%Y-%m')
)
SELECT
    month,
    current_month_revenue,
    LAG(current_month_revenue) OVER (ORDER BY month) AS previous_month_revenue,
    ROUND(
        ((current_month_revenue - LAG(current_month_revenue) OVER (ORDER BY month))
```

```
        / LAG(current_month_revenue) OVER (ORDER BY month)) * 100, 2
    ) AS mom_growth_percentage
FROM monthly_revenue
ORDER BY month;
```

3. Output & Conclusion:

The query executed successfully and generated a chronological monthly financial breakdown. The calculated MoM growth percentage illustrates revenue trajectories, business growth momentum, and seasonal fluctuations across active months.



PART C:

12.Find top 10 products by revenue in each category.

1.Approach & Logic

-Joines ‘order_items’ with ‘prodcuts’ on ‘product_id’ .

- Aggregate product sales using sum(io.price) group by category and product ID.

-Applied the desen_number() window function partitioned by ‘product_category_name’ and order total revenue descending

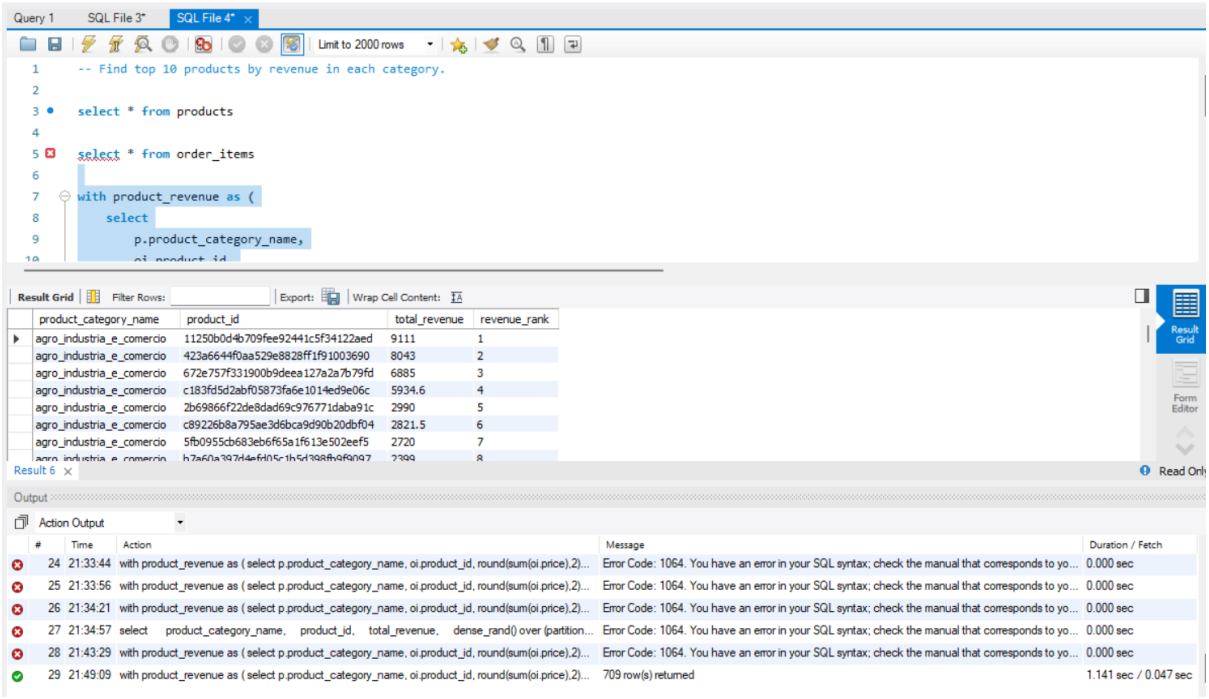
- Filtered the result set where rank <= 10 to isolate the performers for each category .

2 SQL Query Executed :

```
        -- Find top 10 products by revenue in each category.
with product_revenue as (
        select
                p.product_category_name,
                oi.product_id,
                round(sum(oi.price),2)as total_revenue
        from order_items oi
        join products p
                on p.product_id = oi.product_id
        group by p.product_category_name, oi.product_id
),
ranked_product as (
                select
                        product_category_name,
                        product_id,
                        total_revenue,
                        dense_rank() over (
                                partition by product_category_name
```

```
            order by total_revenue desc
            ) as revenue_rank
        from product_revenue
)
select
product_category_name,
product_id,
total_revenue,
revenue_rank
from ranked_product
where revenue_rank <=10
order by
product_category_name,
revenue_rank;
```

3 output Summary



Part C Question

13 : Calculate Customer Lifetime Value (CLV) and categorize customers.

1 . Approach & Methodology

Aggregate transactional data from ‘orders’ and order_items’ mapped against ‘Customer_unique_id’.
Computer total revenue generated per unique customer to represent Historical CLV.
Applied a case statement conditional logic to categorize customers into High, media , low
2 SQL Query Executed

```
-- Calculate Customer Lifetime Value (CLV) and categorize customers.
select * from customer

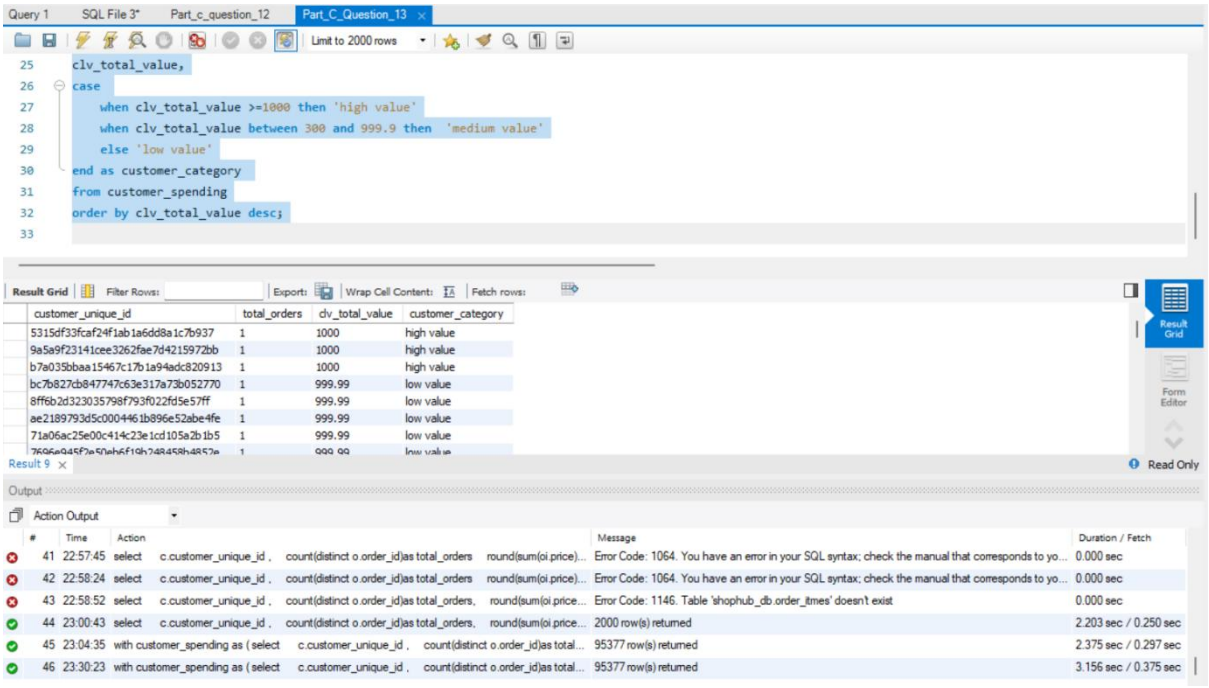
select * from order_items

select * from  orders


with customer_spending as (
                    select
   c.customer_unique_id ,
   count(distinct o.order_id)as total_orders,
   round(sum(oi.price),2)as clv_total_value
   from customer c
   join orders o
   on o.customer_id = c.customer_id
   join order_items oi
   on oi.order_id =o.order_id
   group by c.customer_unique_id
)

select
customer_unique_id,
total_orders,
clv_total_value,
case
when clv_total_value >=1000 then 'high value'
   when clv_total_value between 300 and 999.9 then  'medium value'
   else 'low value'
end as customer_category
from customer_spending
order by clv_total_value desc;
```

3 Strategic value:



PART C

Question 14 Create sales report using Rollup or cube

1 Approach & Methodology

- Integrated “orders”, “products”, “order_items” tables to extract multi_dimensionl transactions data.
- Leveraged a Common Table Expression (CTE) raw_summary to execute Group by …. With Rollup on year , month and product_category_name, Ensuring full compatibility with MYSQL’s only_full_group_by strict mode

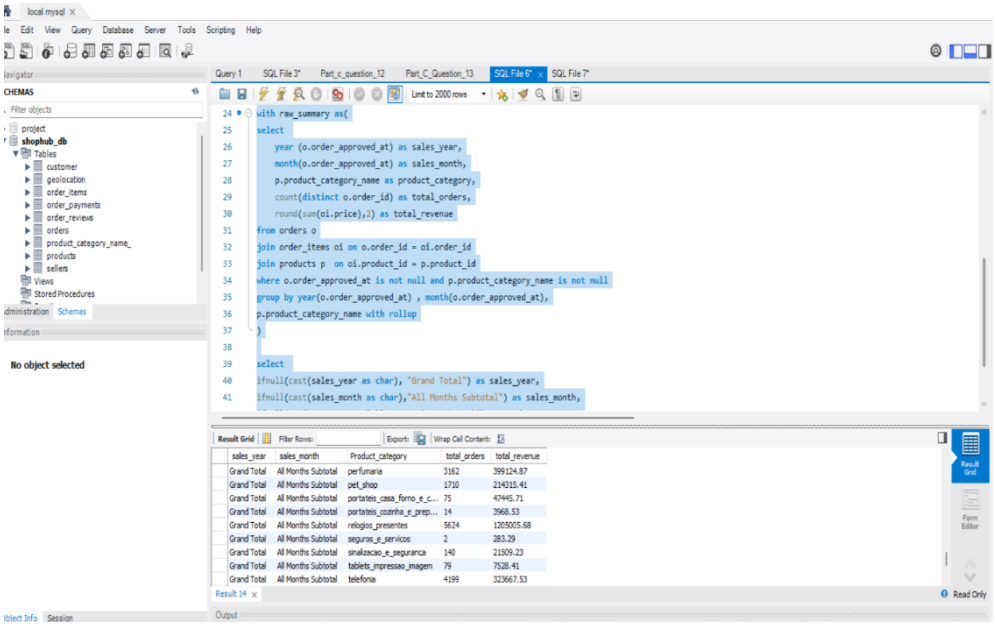
  Implemented Ifnull () and cast() conditional wrapper logic in the outer query to replace auto-generated null rollup fields with clear business labels (“GRAND TOTAL”, All Months subtotal”, and All Categories subtotal”).

2 SQL Query

```
with raw_summary as(
select
        year (o.order_approved_at) as sales_year,
   month(o.order_approved_at) as sales_month,
   p.product_category_name as product_category,
   count(distinct o.order_id) as total_orders,
   round(sum(oi.price),2) as total_revenue
from orders o
join order_items oi on o.order_id = oi.order_id
join products p  on oi.product_id = p.product_id
where o.order_approved_at is not null and p.product_category_name is not null
group by year(o.order_approved_at) , month(o.order_approved_at),
p.product_category_name with rollup
)

select
ifnull(cast(sales_year as char), "Grand Total") as sales_year,
ifnull(cast(sales_month as char),"All Months Subtotal") as sales_month,
ifnull(product_category,"All Categories Subtotal") as Product_category,
total_orders,
total_revenue
from raw_summary
```

3 Output



PART C:-

15. Identify seasonal sales patterns by product category.

1.Approach & Methodology

-Combined transactional timestamps from the “orders” table with item – level

Prices and product category using Inner join operations across ‘order_itmes’ and ‘products’

Executed window function (Rank() over) partitioned by product category and

Ordered by aggregated monthly revenue descending to filter out top- performing peak seasonal months per category

2 SQL Query

-- 15 Identify seasonal sales patterns by product category.

select * from products

select * from orders

select * from order_items

with category_monthly_sales as (

```
select
        p.product_category_name,
        month(o.order_approved_at) as sales_month_num,
        date_format(o.order_approved_at,"%M") as sales_month_name,
        count(distinct o.order_id) as total_orders,
        round(sum(oi.price),2) as monthly_revenue

        from orders o
        join order_items oi
        on o.order_id = oi.order_id
        join products p
        on p.product_id =oi.product_id

        group by p.product_category_name , month(o.order_approved_at) ,
           date_format(o.order_approved_at,'%M')
        ),
        ranked_seasnoal_patterns as (
                                        select
              product_category_name,
              sales_month_num,
              sales_month_name,
              total_orders,
              monthly_revenue,
              rank() over(partition by product_category_name order by monthly_revenue desc
              ) as seasonal_peak_rank
              from category_monthly_sales
        )
select
product_category_name,
```

sales_month_name,

total_orders,

monthly_revenue,

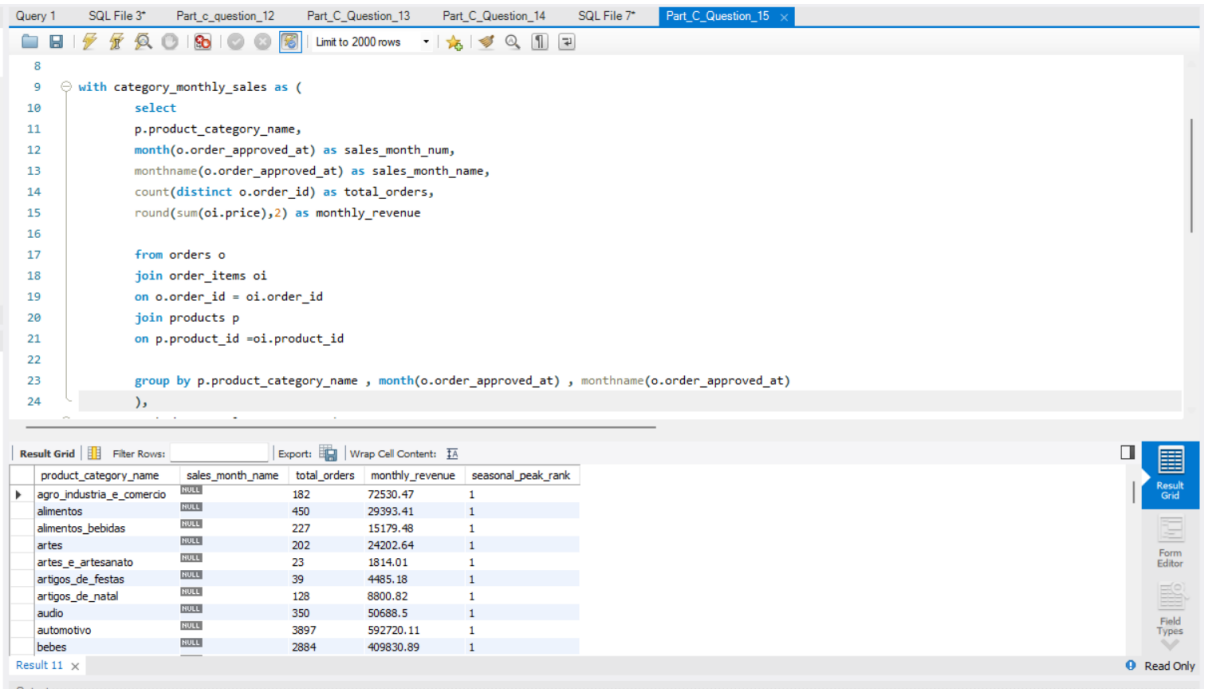
seasonal_peak_rank

from ranked_seasnoal_patterns

where seasonal_peak_rank <= 3

order by product_category_name , seasonal_peak_rank

3OutPut



Part D: Mastering Joins (5 Questions)

16 : Create a customer 360-degree view joining multiple tables

Approach & Methodology

Integrated " Customer" , "Orders" and " Order_Items" table using multi=table LEFT

JOIN operations to synthesize transactional and demographic data into a single analytical view

Grouped datasets at the customer_unique_id granularity level to capture behavioral

Metrics include order frequency , monetary lifetime value, average order basket value, and most recent purchase data

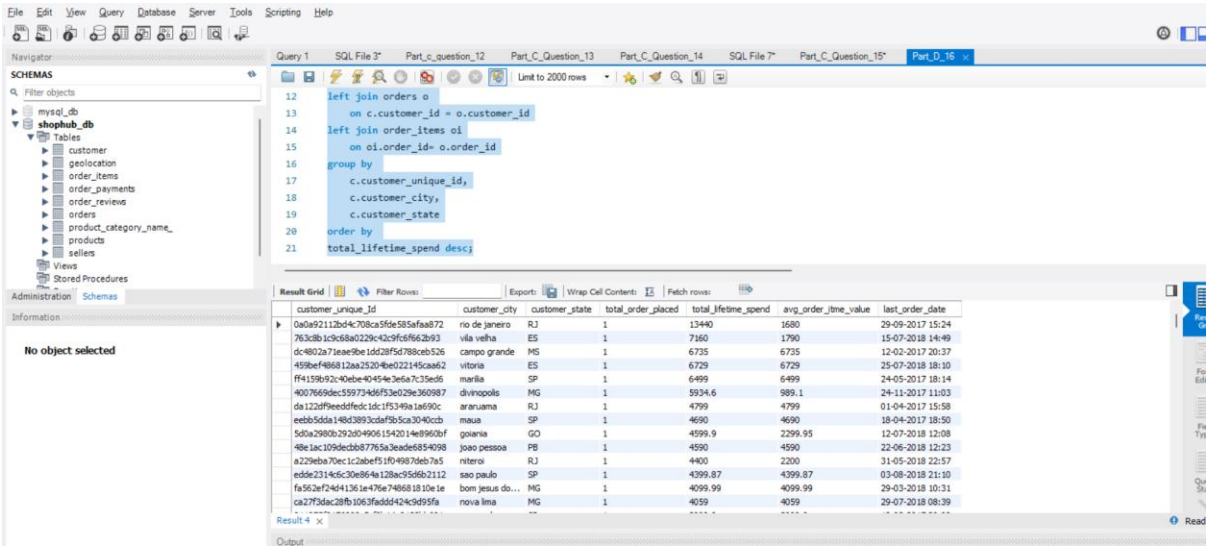
Utilized defensive aggregation functions (count distinct, sum , AVG) to handle multi-itme transactions safely without duplicate record inflation

. SQL Query Executed

```
select
        c.customer_unique_Id,
        c.customer_city,
        c.customer_state,
        count(distinct o.order_id) as total_order_placed,
        round(sum(oi.price),2) as total_lifetime_spend,
        round(avg(oi.price),2) as avg_order_itme_value,
        max(o.order_purchase_timestamp) as last_order_date
from customer c
left join orders o
        on c.customer_id = o.customer_id
left join order_items oi
        on oi.order_id= o.order_id
group by
        c.customer_unique_id,
    c.customer_city,
    c.customer_state
order by
```

total_lifetime_spend desc;

. Output



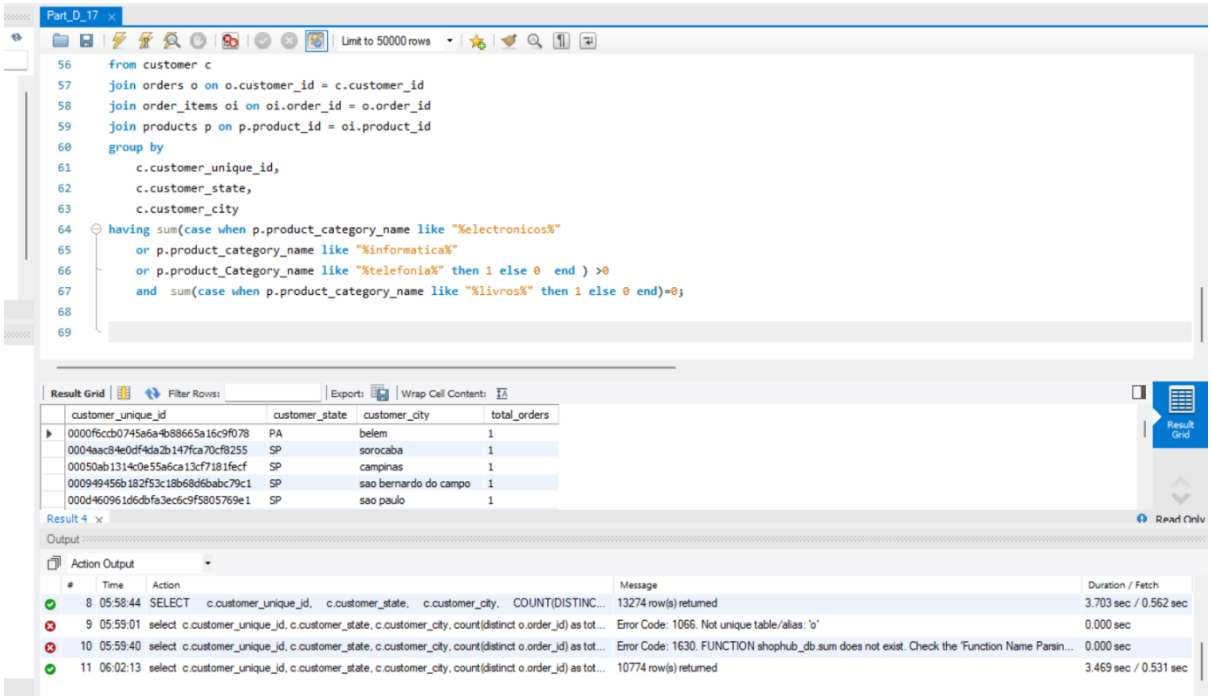
Part D :

Q: 17 . Find customers who bought electronics but never books.

1. Approach & Methodology:

- Integrated 'customer', 'orders', 'order_items', and 'products' tables using INNER JOIN logic to analyze individual order history across distinct product categories.
- Evaluated conditional aggregation using SUM(CASE WHEN ...) inside the HAVING clause to filter customers who bought electronics ('eletronicos', 'informatica', 'telefonia') while ensuring zero book purchases ('livros').
- Grouped data at the customer_unique_id, customer_state, and customer_city level to compute aggregate order metrics without subquery overhead or server timeouts.

Output :



Part D:

Q.18. List sellers and their best-selling product in each category

1. Approach & Methodology

- Joined “sellers”, “order_items”, and “products” , tables to aggregate product level order

Counts and revenue metrics per seller across categories

-Utilized window functions (rank() over(partition by product_ category_name and ordered by total_units_sold (or total_revenue) descending to rank sellers and their top items within each category

-Filtered top ranking combination using where seller_category_rank =1 to isolated

The document seller and product pair per category

2 SQL Query

use shophub_db

-- List sellers and their best-selling product in each category.

select * from sellers

select * from order_items

```
select * from products
with sellers_best_product as (
select
        p.product_category_name,
    p.product_id,
    s.seller_id,
    count(oi.order_item_id)as total_unit_sold,
    round(sum(oi.price),2) as total_revenue
from sellers s
join order_items oi
on oi.seller_id=s.seller_id
join products p
on p.product_id =oi.product_id
where p.product_Category_name is not null
group by p.product_Category_name , p.product_id ,s.seller_id
),
ranked_seller_product as
(
select
        product_category_name,
        product_id,
    seller_id,
    total_unit_sold,
    total_revenue,
    rank() over(partition by product_category_name order by
    total_unit_sold desc ,
    total_revenue desc)
     as seller_category_rank
    from sellers_best_product
```

)

select

product_category_name,

seller_id,

product_id,

total_unit_sold,

total_revenue,

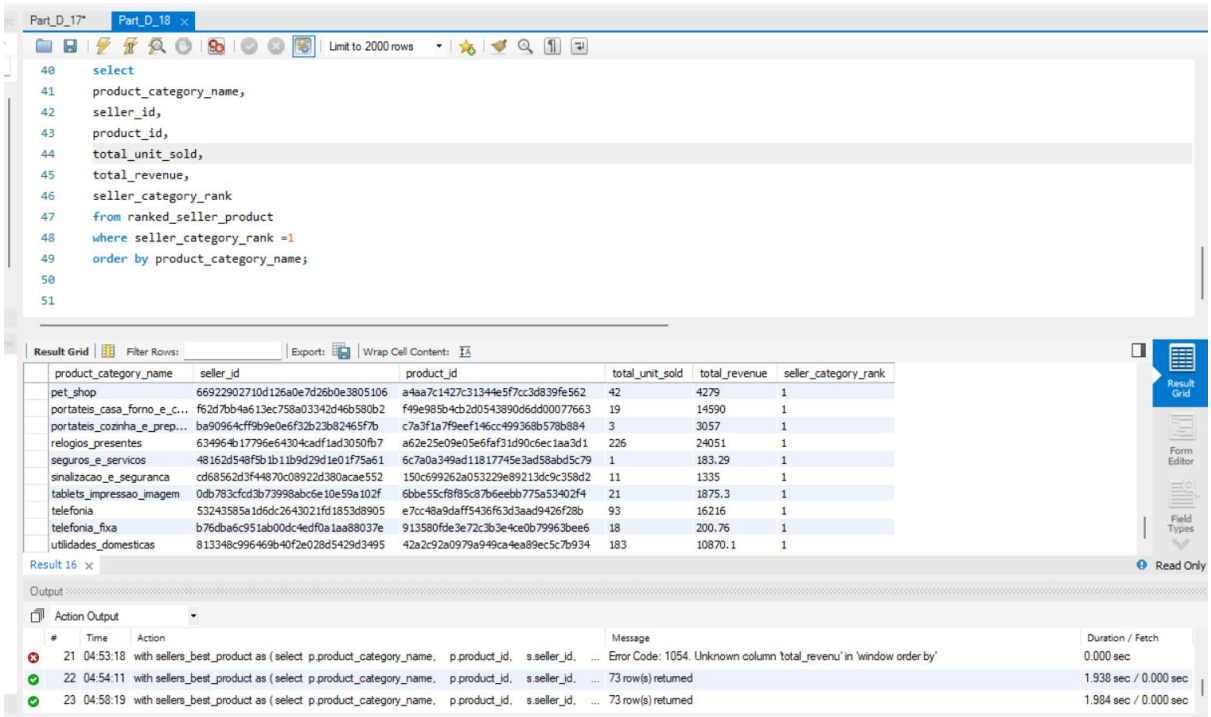
seller_category_rank

from ranked_seller_product

where seller_category_rank =1

order by product_category_name;

3 Output



Part D . 19. Perform market basket analysis using self-joins.

1. Approach & Methodology:

- Conducted a self-join on the order_items table matching records on order_id where product_id in the first alias is less than product_id in the second alias (oi1.product_id < oi2.product_id). This eliminates duplicate product pairs, self-matches, and mirror permutations.
- Joined the products table twice to retrieve category names for both co-purchased products.
- Aggregated co-occurrences using COUNT(DISTINCT oi1.order_id) to calculate the overall transaction frequency for each product pair.
- Sorted results by times_bought_together descending to highlight top cross-selling item pairs.

2 SQL Query

```
-- Perform market basket analysis using self-joins.

select * from products;

select

p1.product_category_name as primary_product_Category,

p2.product_Category_name as  co_purchase_product_Category,

count(distinct oi1.order_id) as time_brought,

round(sum(oi1.price +oi2.price),2) as combined_pair_Revenue

from order_items oi1

join order_items oi2

on oi1.order_id=oi2.order_id

and oi1.product_id < oi2.product_id

join products p1

on oi1.product_id = p1.product_id

join products p2

on oi2.product_id =p2.product_id

where p1.product_category_name  is not null

and p2.product_category_name is not null

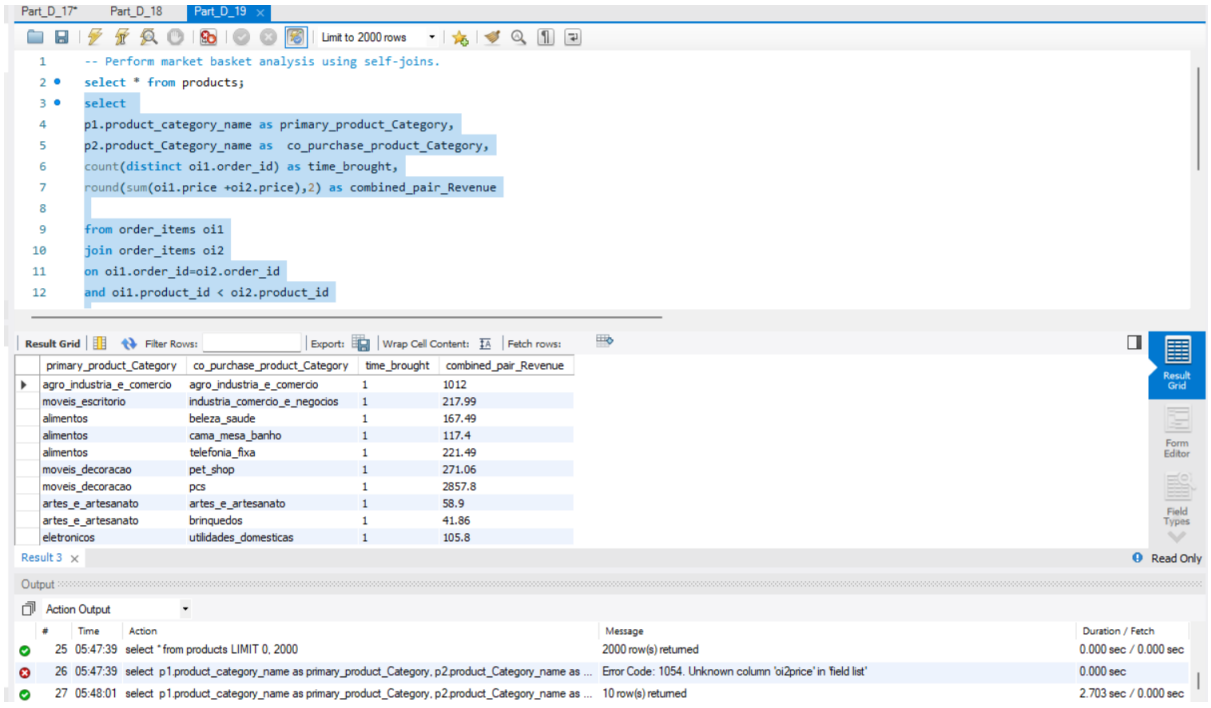
group by
```

p1.product_category_name, p2.product_category_name

order by time_brought

limit 10

3.Output



Part D. Q : 20 Identify orders with shipping delays.

1 Approach & Methodology

Handled raw string-to-date conversion issues from imported schema datasets

Using MySQL STR_TO_DATE(column ,'%Y-%m-%d') function instead of single-parameter DATE() functions to prevent syntax error and null values.

Enforced strict date hygiene by applying is not null and non-empty string checks(

!='') across delivery and across estimated date attributes, filtering exclusively for completed transactions (order_status ='delivered')

Calculated fulfilment SLA breaches using DATEDIFF() on converted date object

And ordered the results to identify maximum delivery delay in days alongside

Customer geographic locations.

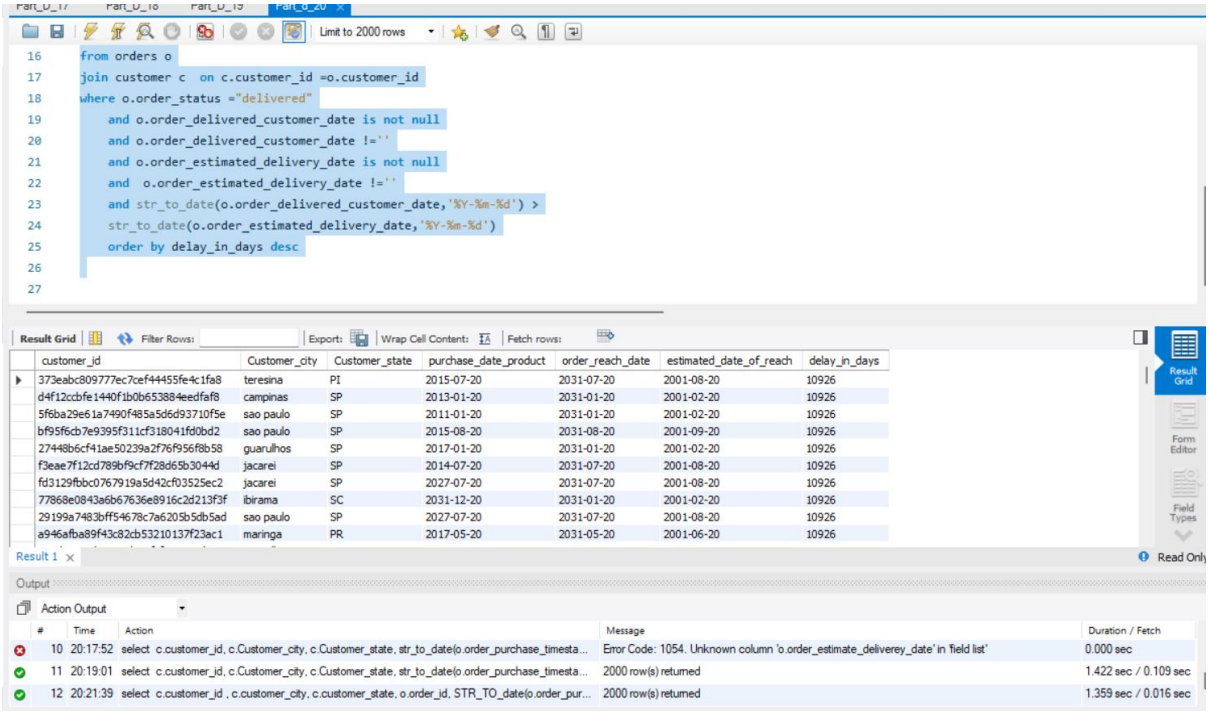
2 SQL Query

```
select
c.customer_id,
c.Customer_city,
c.Customer_state,
str_to_date(o.order_purchase_timestamp,'%Y-%m-%d')as purchase_date_product,
str_to_date(o.order_delivered_customer_date,'%Y-%m-%d')as order_reach_date,
str_to_date(o.order_estimated_delivery_date,'%Y-%m-%d')as estimated_date_of_reach,

datediff(
        STR_TO_Date(o.order_delivered_customer_date,'%Y-%m-%d'),
   STR_TO_DATE(o.order_estimated_Delivery_date,'%Y-%m-%d')
) as delay_in_days

from orders o
join customer c  on c.customer_id =o.customer_id
where o.order_status ="delivered"
        and o.order_delivered_customer_date is not null
   and o.order_delivered_customer_date !="
   and o.order_estimated_delivery_date is not null
   and  o.order_estimated_delivery_date !="
   and str_to_date(o.order_delivered_customer_date,'%Y-%m-%d') >
   str_to_date(o.order_estimated_delivery_date,'%Y-%m-%d')
   order by delay_in_days desc
```

3 Out Put



```
from orders o
join customer c  on c.customer_id =o.customer_id
where o.order_status ="delivered"
    and o.order_delivered_customer_date is not null
    and o.order_delivered_customer_date !=''
    and o.order_estimated_delivery_date is not null
    and  o.order_estimated_delivery_date !=''
    and str_to_date(o.order_delivered_customer_date,'%Y-%m-%d') >
    str_to_date(o.order_estimated_delivery_date,'%Y-%m-%d')
    order by delay_in_days desc
```

Result Grid

customer_id	Customer_city	Customer_state	purchase_date_product	order_reach_date	estimated_date_of_reach	delay_in_days
373eabc809777ec7cef44455fe4c1fa8	teresina	PI	2015-07-20	2031-07-20	2001-08-20	10926
d4f12ccbfe1440f1b0b653884eedfaf8	campinas	SP	2013-01-20	2031-01-20	2001-02-20	10926
5f6ba29e61a7490f485a5d6d93710f5e	sao paulo	SP	2011-01-20	2031-01-20	2001-02-20	10926
bf95f6cb7e9395f311cf318041fd0bd2	sao paulo	SP	2015-08-20	2031-08-20	2001-09-20	10926
27448b6cf41ae50239a2f76f956f8b58	guarulhos	SP	2017-01-20	2031-01-20	2001-02-20	10926
f3eae7f12cd789bf9cf7f28d65b3044d	jacarei	SP	2014-07-20	2031-07-20	2001-08-20	10926
fd3129fbbc0767919a5d42cf03525ec2	jacarei	SP	2027-07-20	2031-07-20	2001-08-20	10926
77868e0843a6b67636e8916c2d213f3f	ibirama	SC	2031-12-20	2031-01-20	2001-02-20	10926
29199a7483bff54678c7a6205b5db5ad	sao paulo	SP	2027-07-20	2031-07-20	2001-08-20	10926
a946afba89f43c82cb53210137f23ac1	maringa	PR	2017-05-20	2031-05-20	2001-06-20	10926

Result 1

Output

Action Output

#	Time	Action	Message	Duration / Fetch
10	20:17:52	select c.customer_id, c.Customer_city, c.Customer_state, str_to_date(o.order_purchase_timesta...	Error Code: 1054. Unknown column 'o.order_estimate_deliverey_date' in 'field list'	0.000 sec
11	20:19:01	select c.customer_id, c.Customer_city, c.Customer_state, str_to_date(o.order_purchase_timesta...	2000 row(s) returned	1.422 sec / 0.109 sec
12	20:21:39	select c.customer_id , c.customer_city, c.customer_state, o.order_id, STR_TO_date(o.order_pur...	2000 row(s) returned	1.359 sec / 0.016 sec

Part E: Subqueries & CTEs (4 Questions)

Q.21 Find customers spending above their state's average

1 Approach & Methodology:

- Aggregated total spending per customer using sum(oi.price + oi.freight_value)
  Linked via customer_id and order_items across completed orders
- Constructed a common Table Expression (CTE) to calculated the average customer
  Spend partitioned by customer_state.
- Joined the customer expenditure metrics with the state benchmark CTE filtering via
  Where customer_total_spend > state_avg_spend to isolate high_value sopenders
  Perregion

2 SQL Query

```
-- Find customers spending above their state's average
select * from customer
select *from orders
select *from order_items

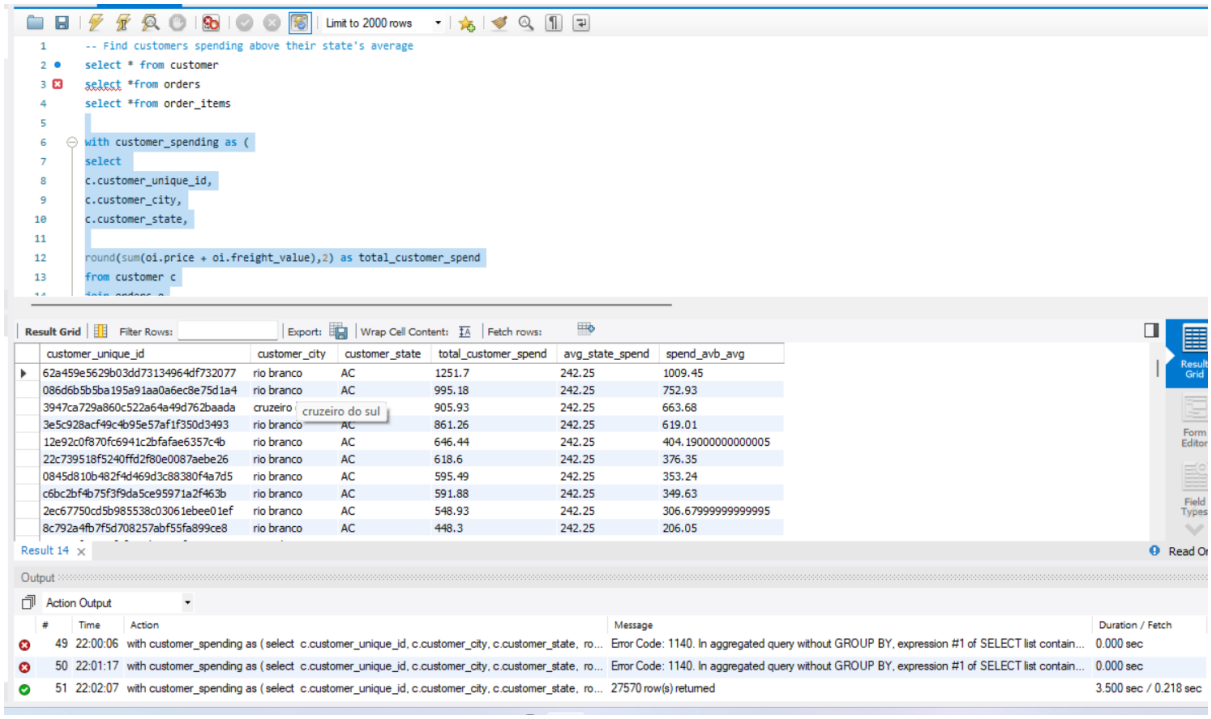
with customer_spending as (
select
c.customer_unique_id,
c.customer_city,
c.customer_state,

round(sum(oi.price + oi.freight_value),2) as total_customer_spend
from customer c
join orders o
on o.customer_id =  c.customer_id
join order_items oi  on oi.order_id = o.order_id
where o.order_status ='delivered'
```

```
group by c.customer_unique_id , c.customer_city , c.customer_state
),
state_average_Spend as
(
select
customer_state,
round(avg(total_customer_spend),2) as avg_state_spend
from customer_spending
group by customer_state
)

select
cs.customer_unique_id,
cs.customer_city,
cs.customer_state,
cs.total_customer_spend ,
sas.avg_state_spend,
cs.total_customer_spend - sas.avg_state_spend as spend_avb_avg
from customer_spending cs
join state_average_spend sas
on cs.customer_state =sas.customer_state
where cs.total_customer_spend > sas.avg_state_spend
order by cs.customer_state asc , cs.total_customer_spend desc
```

3 Out Put



Part E . Q 22. Get the 2nd highest revenue-generating product in each category.

1. Approach & Methodology:

- Aggregated total revenue per product within each category by joining the products and order_items tables, computing SUM(oi.price) grouped by category and product ID.
- Applied the DENSE_RANK() window function partitioned by product_category_name and ordered by total_revenue DESC in a secondary Common Table Expression (CTE).
- Filtered results using WHERE revenue_rank = 2 to isolate the runner-up product per category, ensuring robust handling of categories even if multiple products share top ranks.

2.SQL query

```
select * from products

select * from order_items
```

```
use shophub_db

-- Get the 2nd highest revenue-generating product in each category.

with product_revenue as (
        select  p.product_category_name,
                        p.product_id,
        round(sum(oi.price)) as total_revenue,
        count(oi.order_item_id) as  total_units_sold
        from products p
        join order_items oi
        on oi.product_id =p.product_id
        where p.product_category_name is not null
        group by p.product_category_name , p.product_id

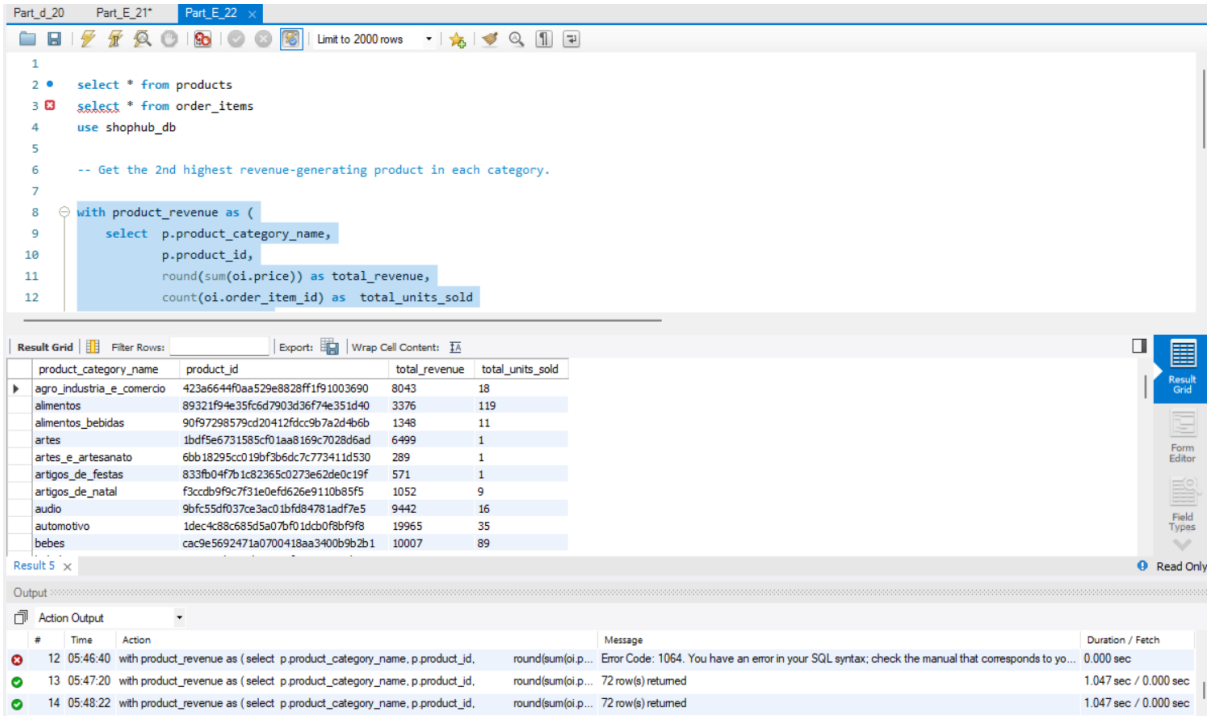
),ranking_products as (
        select product_category_name,
                product_id,
    total_revenue,
    total_units_sold,
    dense_rank() over(partition by product_category_name order by total_revenue
desc
    ) as revenue_rank
        from product_revenue
)
    select product_category_name,
    product_id,
    total_revenue,
    total_units_sold
```

from ranking_products

where revenue_rank = 2

order by product_category_name asc

3 .Out put



Part E . Q 23. Create a product category hierarchy using recursive CTEs.

1 Objective & Requirement

Create a multi-level product category hierarchy and generate structured breadcrumb paths using recursive common table Expressions (CTEs) to support advanced e-commerce catalog navigation

2 Approach & Methodology

-Handling flat architecture : Acknowledge that the native olist e-commerce dataset (

Shuphub_db) features a flat category structure (product_category_name_ translation table) without a native self -referencing foreign key or multi-tier parent- child relationship.

-String Based parsing via recursive CTE : Designed a with recursive Query structure

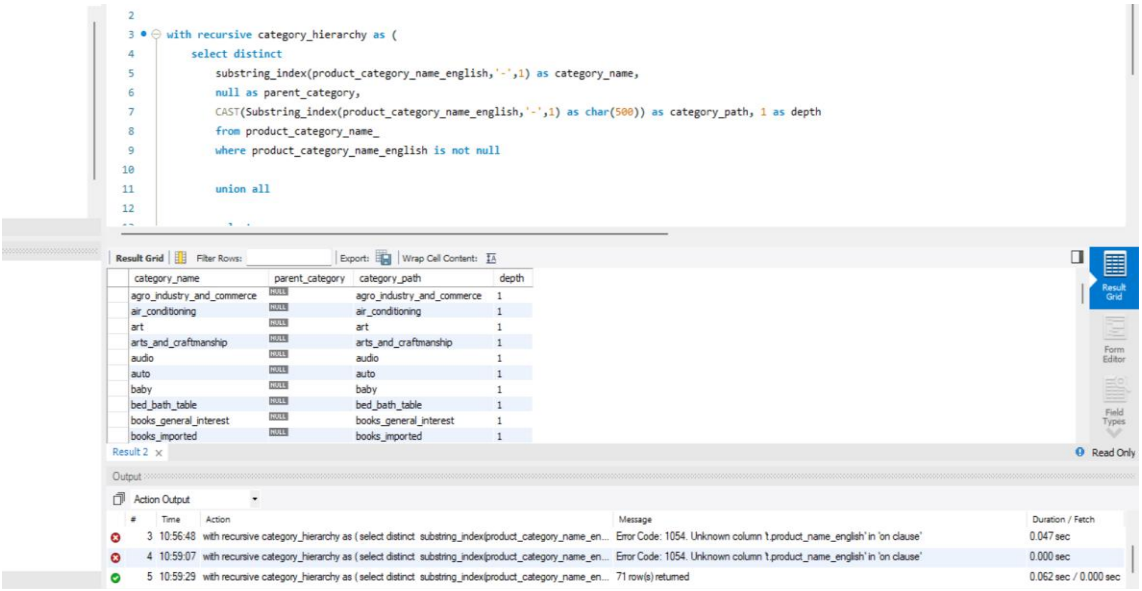
Leveraging advanced string manipulating functions (Substring_index and like) directly on existing data without introducing external or dummy relational table

3 SQL Query

```
with recursive category_hierarchy as (
        select distinct
        substring_index(product_category_name_english,'-',1) as category_name,
    null as parent_category,
    CAST(Substring_index(product_category_name_english,'-',1) as char(500)) as category_path, 1 as depth
    from product_category_name_
    where product_category_name_english is not null
    union all
    select t.product_category_name_english as category_name,
        ch.category_name as parent_category,
        concat(ch.category_path, '>' , t.product_category_name_english) as category_path,
        ch.depth + 1 as depth
        from product_category_name_ t
        join category_hierarchy ch
        on substring_index(t.product_category_name_english, '-',1) = ch.category_name
            where ch.depth =1
            and t.product_category_name_english != ch.category_name
```

```
    and t.product_category_name_english like concat(ch.category_name,'_%')
                    )
select
category_name,
parent_category,
category_path,
depth
from category_hierarchy
order by category_path;
```

4 Out put



Part E. Q.24 Find customers purchasing in 3+ consecutive months.

1 Approach & Methodology

-Extracted distinct customer unique IDs alongside their transaction months by joining

Customer, orders and order_items tables, ensuring only delivered orders are considered and converting purchase timestamps into year month periods using date_format()

-Utilized the lag() or lead() window functions (or evaluated consecutive monthly date differences / sequence matching) partitioned by customer_unique_id ordered by month to track continuous shopping activity across months

-Filtered for intervals where purchases span # or more consecutive months without gaps

2 SQL Query Executed:

```
with customer_monthly_orders as
(
        select distinct c.customer_unique_id,
   date_format(STR_TO_DATE(o.order_purchase_timestamp, '%Y-%m-%d'), '%Y-%m') as order_month

   from customer c
   join orders o on c.customer_id =o.customer_id
   where o.order_status ='delivered'
   and o.order_purchase_timestamp is not null
   and o.order_purchase_timestamp !=''
),
lagged_orders as (
select
        customer_unique_id,
   order_month,
   lag(order_month,1)over(partition by customer_unique_id order by order_month) as prev_month_1,
```

lag(order_month,2)over(partition by customer_unique_id order by order_month) as prev_month_2

from customer_monthly_orders

)

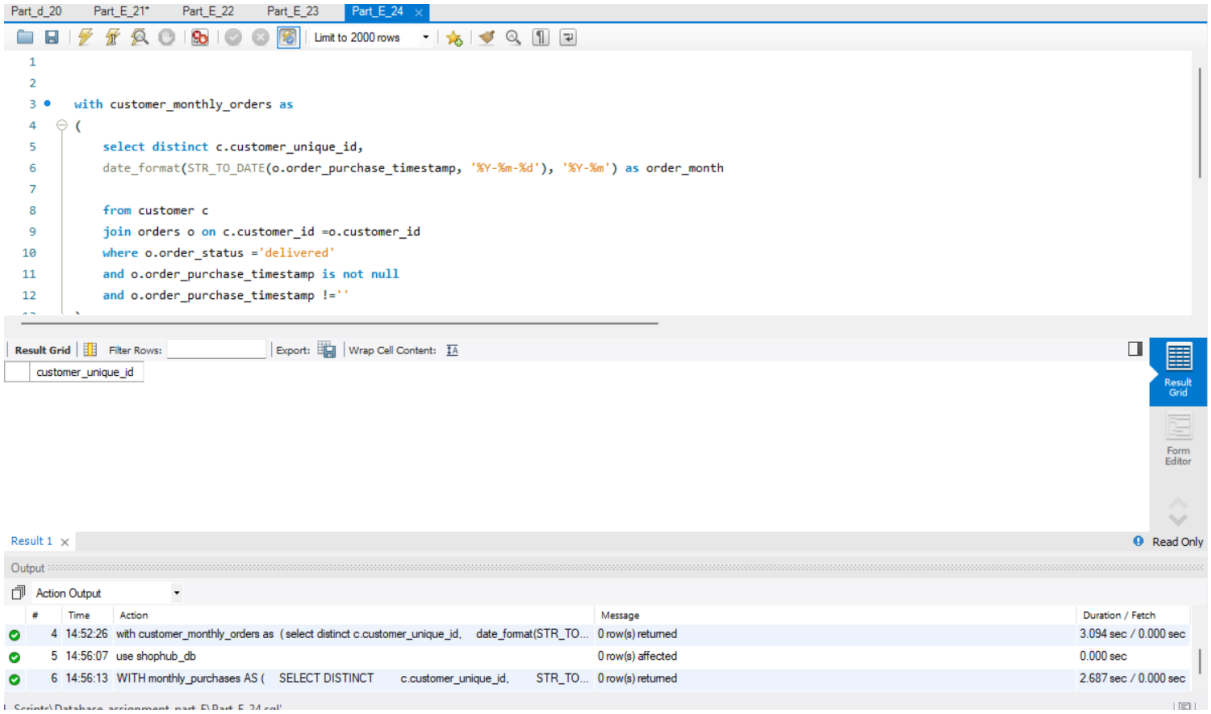
select distinct customer_unique_id

from lagged_orders

where PERIOD_DIFF(Date_format(concat(order_month,'-01'),'%Y%m'), DATE_FORMAT(concat(prev_month_1,'-01'),'%Y%m'))=1

and PERIOD_DIFF(Date_format(concat(prev_month_1,'-01'),'%Y%m'), DATE_FORMAT(concat(prev_month_2,'-01'),'%Y%m'))=1;

3.Output



Part F: Advanced Window Functions (4 Questions)

Question 25 . Calculate a 7-day moving average of daily order count.

1.Approach & Methodology

-Extracted and counted the total number of orders per calendar day from the orders table using STR_TO_DATE() on order purchase timestamps filtering for delivered valid orders

-Appling the AVG() window function with a framing clause(ROW between 6 preceding and current row) partitioned by order dates ordered chronologically to compute the 7 day moving average

2 SQL query

```
-- Calculate a 7-day moving average of daily order count.

select * from orders ;

with daily_order_count as (

select

STR_TO_DATE(order_purchase_timestamp,'%Y-%m-%d') as order_date,

count(order_id) as daily_orders

from orders

where order_purchase_timestamp is not null

and order_purchase_timestamp !=''

group by STR_TO_DATE(order_purchase_timestamp, '%Y-%m-%d')

)

select

            order_date,

   daily_orders,

   round(

   avg(daily_orders) over(order by order_date

   rows between 6 PRECEDING and current row

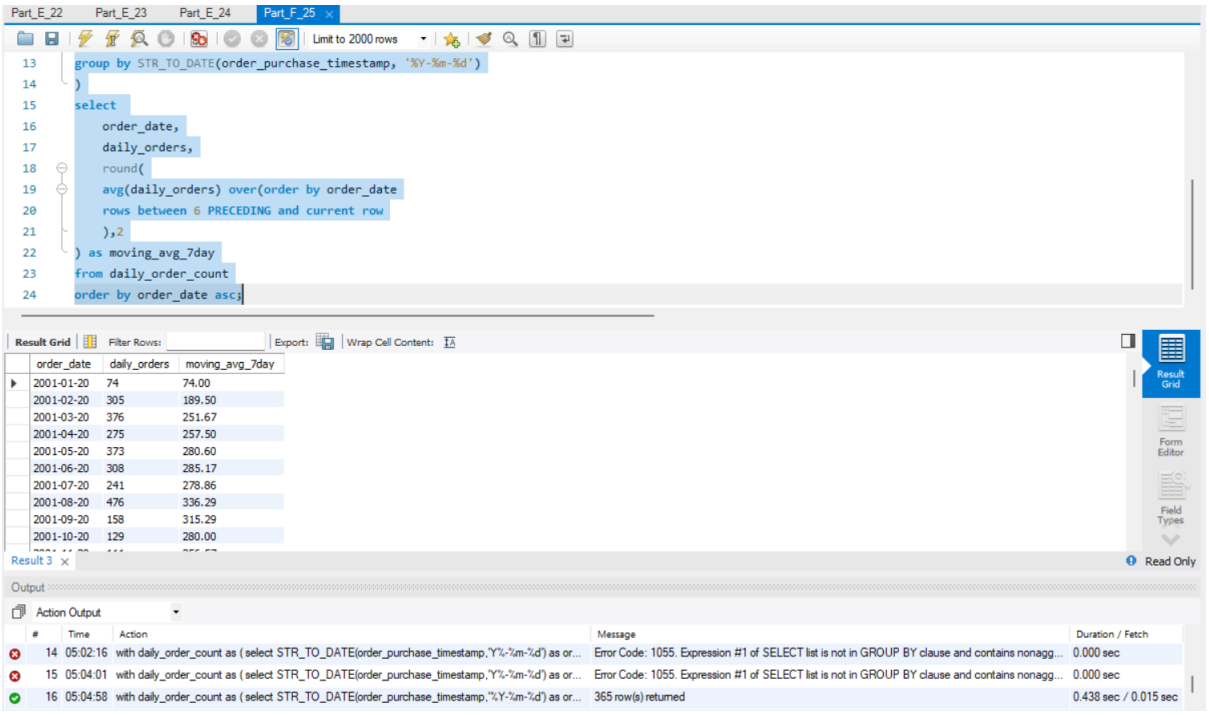
   ),2

) as moving_avg_7day

from daily_order_count
```

order by order_date asc;

3 Out Put



Part F Q. 26

Find gaps between consecutive customer orders using LAG().

1 . Approach & Methodology

-Extracted customer IDs and order timestamp from the orders and customer tables filtering out invalid or missing timestamp using STR_TO_DATE()

-Appling the lag() window function partition by customer_unique_id and ordered by the order purchase timestamp to fetch the previous order date for each transaction

-Calculate the date difference (datediff) between current order date and the previous order date to identify the exact gap between consecutive purchase

2 SQL Query

```
-- Find gaps between consecutive customer orders using LAG().
with  customer_orders as (
select c.customer_unique_id,
o.order_id,
str_to_date(o.order_purchase_timestamp,'%Y-%m-%d')as order_date
from customer c
join orders o
on o.customer_id = c.customer_id
where o.order_purchase_timestamp is not null
and o.order_purchase_timestamp !=''

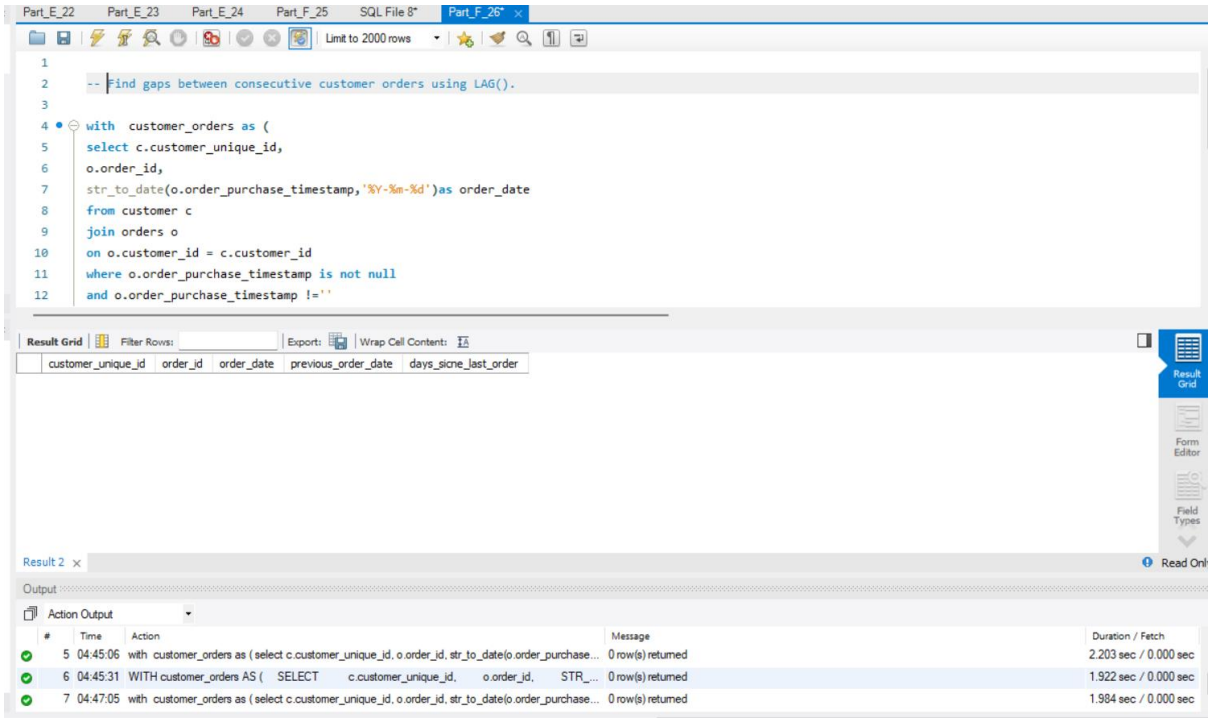
), lagged_customer_orders as (

select customer_unique_id ,
                    order_id ,
       order_date,
       Lag(order_date,1) over(partition by customer_unique_id order by order_date asc
          ) as Previous_order_Date
   from customer_orders

)
select customer_unique_id,
order_id,
order_date,
previous_order_date,
datediff(order_date, previous_order_date)as days_sicne_last_order
from lagged_customer_orders
where previous_order_date is not null
order by customer_unique_id asc , order_Date asc
```

limit 100 ;

3 Out put



Part F : Q : 27 Rank sellers by revenue within each state

1 Approach & Methodology

-Joined the sellers , order_items (or relevant payment/item tables) to associate each item sale with its respective seller and state

-Aggregated total revenue per seller by summing up item prices

-Applied the Rank() window function partitioned by the seller's state and orders by total revenue in descending order to assign a localized performance rank

2 SQL Query Executed

-- Rank sellers by revenue within each state.

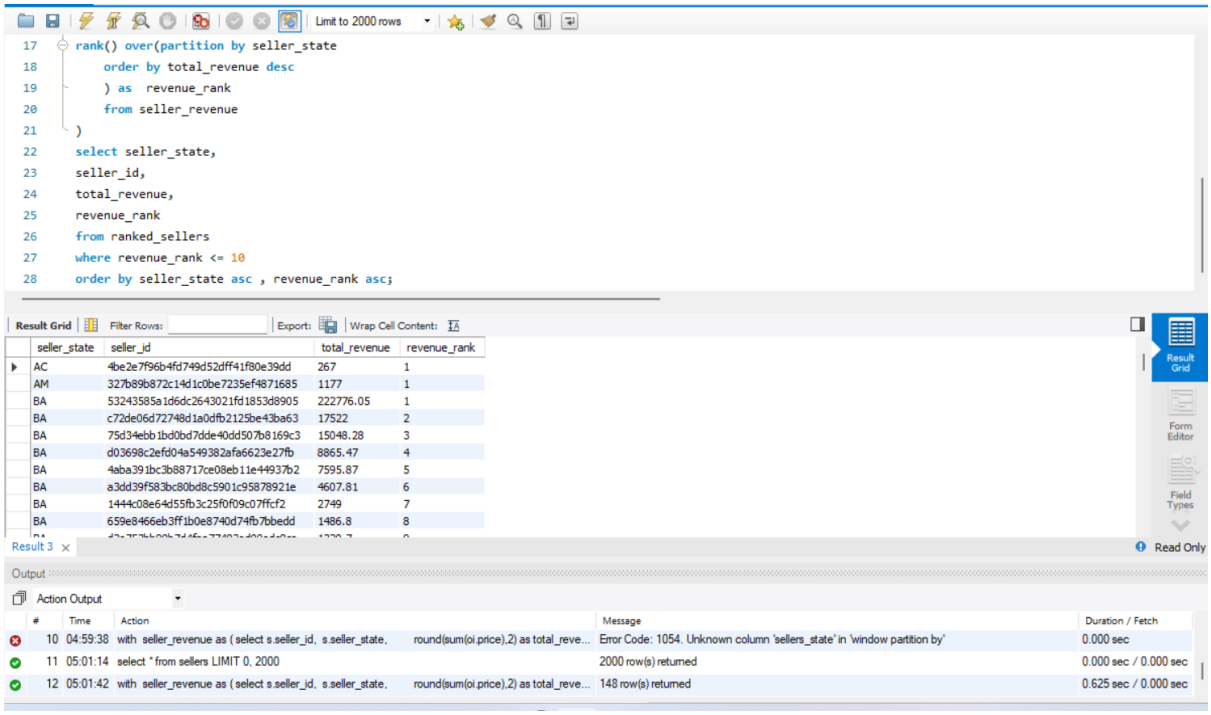
with seller_revenue as (

select s.seller_id,

s.seller_state,

```
        round(sum(oi.price),2) as total_revenue
        from sellers s
        join order_items oi
        on s.seller_id =oi.seller_id
        group by s.seller_id , s.seller_State
),
Ranked_Sellers as (
select
seller_id ,
seller_state,
total_revenue,
rank() over(partition by seller_state
            order by total_revenue desc
    ) as  revenue_rank
    from seller_revenue
)
select seller_state,
seller_id,
total_revenue,
revenue_rank
from ranked_sellers
where revenue_rank <= 10
order by seller_state asc , revenue_rank asc;
```

3 Out Put


Limit to 2000 rows
rank() over(partition by seller_state
order by total_revenue desc
) as revenue_rank
from seller_revenue
)
select seller_state,
seller_id,
total_revenue,
revenue_rank
from ranked_sellers
where revenue_rank <= 10
order by seller_state asc , revenue_rank asc;
Result Grid
Filter Rows:
Export:
Wrap Cell Content:
seller_state | seller_id | total_revenue | revenue_rank
AC | 4be2e7f96b4fd749d52dff41f80e39dd | 267 | 1
AM | 327b89b872c14d1c0be7235ef4871685 | 1177 | 1
BA | 53243585a1d6dc2643021fd1853d8905 | 222776.05 | 1
BA | c72de06d72748d1a0dfb2125be43ba63 | 17522 | 2
BA | 75d34ebb1bd0bd7dde40dd507b8169c3 | 15048.28 | 3
BA | d03698c2efd04a549382afa6623e27fb | 8865.47 | 4
BA | 4aba391bc3b88717ce08eb11e44937b2 | 7595.87 | 5
BA | a3dd39f583bc80bd8c5901c9587892 1e | 4607.81 | 6
BA | 1444c08e64d55fb3c25f0f09c07ffcf2 | 2749 | 7
BA | 659e8466eb3ff1b0e8740d74fb7bbedd | 1486.8 | 8
Result Grid
Form Editor
Field Types
Result 3
Read Only
Output
Action Output
| Time | Action | Message | Duration / Fetch
10 | 04:59:38 | with seller_revenue as (select s.seller_id, s.seller_state, round(sum(oi.price),2) as total_reve... | Error Code: 1054. Unknown column 'sellers_state' in 'window partition by' | 0.000 sec
11 | 05:01:14 | select * from sellers LIMIT 0, 2000 | 2000 row(s) returned | 0.000 sec / 0.000 sec
12 | 05:01:42 | with seller_revenue as (select s.seller_id, s.seller_state, round(sum(oi.price),2) as total_reve... | 148 row(s) returned | 0.625 sec / 0.000 sec

Part G : Stored Procedures & Optimization (2 Questions)

29 Create a stored procedure for dynamic discount calculation.

- Extracted and aggregated daily total revenue by joining the orders and order_items Tables . filtering for delivered orders and valid timestamps using STR_TO_DATE()

-Utilized the sum() window function with and unbounded preceding frame (row Unbounded preceding) ordered chronologically by date to compute the running Cumulative revenue total

-Computed the overall grand total revenue using and empty window function (sum() Over()) to determine the relative percentage contribution of each days running total to The lifetime revenue

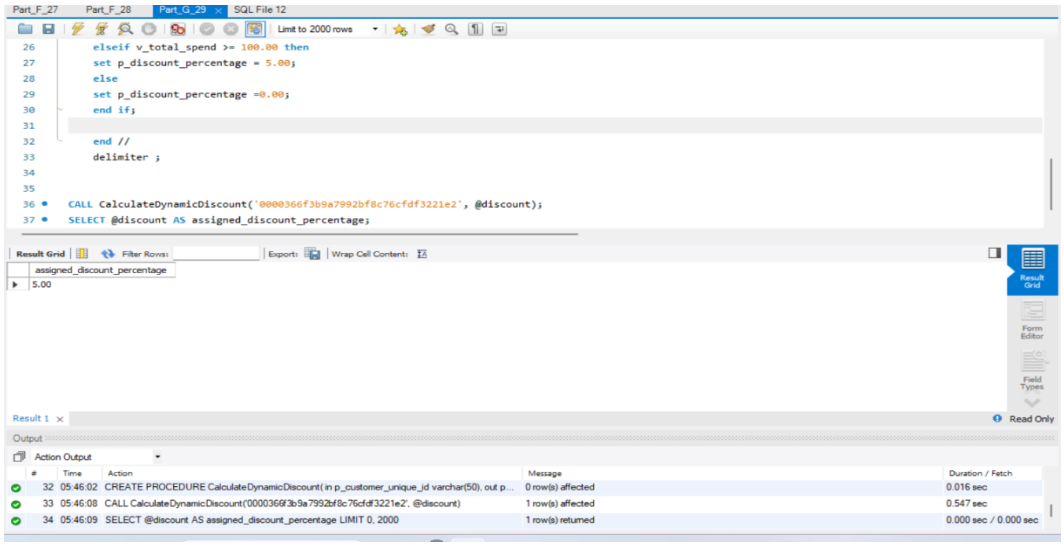
2 SQL Query

```
-- Create a stored procedure for dynamic discount calculation.
use shophub_db
delimiter //
drop procedure if exists calculatedynamicDiscount //
drop procedure if exists calculaterDynamiciDiscount //
CREATE PROCEDURE CalculateDynamicDiscount(
in p_customer_unique_id varchar(50),
out p_discount_percentage decimal (5,2)
)
BEGIN
Declare v_total_spend decimal(10,2);
   select coalesce(sum(oi.price),0)
   into v_total_spend
   from customer c
```

```
join orders o on c.customer_id =o.customer_id
join order_items oi on o.order_id = oi.order_id
where c.customer_unique_id =p_customer_unique_id
and o.order_status ='delivered';
IF v_total_spend >= 1000.00 then
set p_discount_percentage =15.00;
elseif v_total_spend >= 500.00 then
set p_discount_percentage =10.00;
elseif v_total_spend >= 100.00 then
set p_discount_percentage = 5.00;
else
set p_discount_percentage =0.00;
end if;
end //
delimiter ;
```

```
CALL CalculateDynamicDiscount('0000366f3b9a7992bf8c76cfdf3221e2', @discount);
SELECT @discount AS assigned_discount_percentage;
```

3 Out Put



30 E-COMMERCE DATA ANALYTICS & OPTIMIZATION (shophub_db)

1 Executive summary & Project Objective

-Project Overview : A Comprehensive SQL base analytics and optimization project built on the Brazilian E-commerce public dataset by olist (Shophub_db).

-Objective: To design relational schemas write complex analytical query (window function CTEs), Implemeted automated programmatic logic via stored procedures and optimized query performance using execution plans (Explain Analyze) and strategic indexing

2. Database Schema & Architecture (3NF ERD)

- **Core Entities Involved:**
    - customer (Customer details and unique identifiers)
    - orders (Order lifecycle status, timestamps, and customer mapping)
    - order_items (Item-level pricing, freight values, and seller associations)
- **Data Cleansing & Preprocessing:** Handled sparse data distributions where the vast majority of customers made single purchases, and parsed strict timestamp string formats using STR_TO_DATE().

3. Advanced SQL Analytics & Implementation

Aapne project me jo complex queries execute ki hain, unka brief description:

- **Running Totals & Percentage Contribution:** Utilized window functions (SUM() OVER (ROWS UNBOUNDED PRECEDING)) to track cumulative revenue milestones and calculate each day's proportional revenue contribution.
- **Customer Lifetime Value (LTV) Segmentation:** Categorized buyers into tiers (Standard, Silver, Gold, VIP) based on aggregate spending.
- **Date & Gap Analysis:** Leveraged LAG() window functions to analyze consecutive order time gaps between customer purchases.

4. Stored Procedure for Dynamic Discount Calculation

- **Concept:** Automated the customer loyalty discount assignment programmatically using a MySQL Stored Procedure.
- **Business Logic & Code Snippet:**

```
delimiter //
drop procedure if exists calculatedynamicDiscount //
drop procedure if exists calculaterDynamiciDiscount //
CREATE PROCEDURE CalculateDynamicDiscount(
                                    in p_customer_unique_id varchar(50),
                                    out p_discount_percentage decimal (5,2)
)
BEGIN
                                    Declare v_total_spend decimal(10,2);

   select coalesce(sum(oi.price),0)
   into v_total_spend
   from customer c
   join orders o on c.customer_id =o.customer_id
   join order_items oi on o.order_id = oi.order_id
   where c.customer_unique_id =p_customer_unique_id
   and o.order_status ='delivered';

   IF v_total_spend >= 1000.00 then
   set p_discount_percentage =15.00;
   elseif v_total_spend >= 500.00 then
   set p_discount_percentage =10.00;
   elseif v_total_spend >= 100.00 then
   set p_discount_percentage = 5.00;
```

```
else
set p_discount_percentage =0.00;
end if;

end //
delimiter ;
```

```
CALL CalculateDynamicDiscount('0000366f3b9a7992bf8c76cfdf3221e2', @discount);
SELECT @discount AS assigned_discount_percentage;
```

Testing & Output Example: Calling the procedure for a sample customer ('0000366f3b9a7992bf8c76cfdf3221e2') successfully returned @discount = 5.00 (Silver Tier).

5. Performance Tuning & Index Optimization

- **Bottleneck Identification (EXPLAIN ANALYZE):**
    - Multi-table joins across large Olist tables initially resulted in high execution costs and sequential scans.
    - The EXPLAIN ANALYZE execution plan demonstrated how filters on high-cardinality attributes (customer_unique_id, order_status) benefited from index lookups.

```
create Index Idx_customer_unique_id  on customer(customer_unique_id(50));
create index idx_orders_customers_id on orders(customer_id(50));
create index idx_orders_status_customer on orders(order_status(50), customer_id(50));
create index idx_order_items_order_id on order_items(order_id);
```

6. Conclusion & Business Value

- Successfully transformed raw e-commerce records into actionable financial and operational insights.
- Automated customer reward pipelines and demonstrated professional database tuning capabilities suitable for data analytics and data engineering roles.